The pathwidth complex

Alexandre Linhares
affiliation to be added

Horacio Hideki Yanasse
affiliation to be added

Draft, 11 October 2026

Abstract

In 2001 and 2002 we listed twelve problems from operations research, VLSI design and graph theory, the minimization of open stacks problem (MOSP) among them, as equal "plus or minus one", and gave no proofs (Linhares, 2001; Linhares and Yanasse, 2002). We show that most of them are, mathematically, one quantity, *graph pathwidth*, studied independently by communities that rarely cite one another, and we correct our table. Eight rows are exact, counting pathwidth itself: each equals pathwidth or pathwidth plus one. Two lie in *bands*: they stay within one or two of pathwidth, but not at a fixed offset. Two are false as stated. We add three members the table missed, among them the maximum wavefront of sparse matrix elimination, which brings a fourth discipline, numerical linear algebra, into the family. Every relation is machine-checked in the Lean proof assistant. We call this cluster the *pathwidth complex*, as the literature uses the name *pathwidth* more than three times as often as the other eleven names combined. Since 2005, the next most active name is MOSP, and the open-stacks community produced, by way of that year's Constraint Modelling Challenge, the best exact algorithm for MOSP: the *customer search* of Chu and Stuckey (2009), which branches on the order in which customers' stacks close, and which is also a pathwidth algorithm. We prove in Lean that two of its published dominance theorems, rules that discard a branch as no better than another, are false, and that on a 34-customer instance their algorithm, with its better move off, reports an optimum of 7 where the optimum is 6. Our corrected search and a SAT solver with a checked DRAT proof confirm the 6. Our corrected definite move, the first of the two, was also found by Tamaki as a lemma (Kitsunai et al., 2016, Lemma 1); we test it with one bipartite matching per candidate and prove our corrected search sound. We re-prove all 115 benchmark optima that rested on Chu and Stuckey's algorithm, changing none. Ported to graphs, our corrected search proves, within 600 seconds per graph, the pathwidth of 11,194 of the 11,534 Rome graphs, a standard graph-drawing benchmark of 10 to 110 vertices, and, on every benchmark set with published exact counts, as many graphs or more than the published branch and bound of Coudert, Mazauric and Nisse (2016). In the other direction, from pathwidth to MOSP, a lower bound from graph minors, proved in Lean, settles in seconds instances whose refutation took core-days, and closes the Mycielski graph M_7. Our proofs and our solvers are open source. We also release a dataset of 17,714 graphs, distinct up to isomorphism, from twenty benchmark collections for pathwidth and the problems equal to it, the MOSP instances and the Rome graphs among them, 16,089 of them with a certified pathwidth, with independent checkers. We believe ours is currently the strongest exact pathwidth solver openly available.

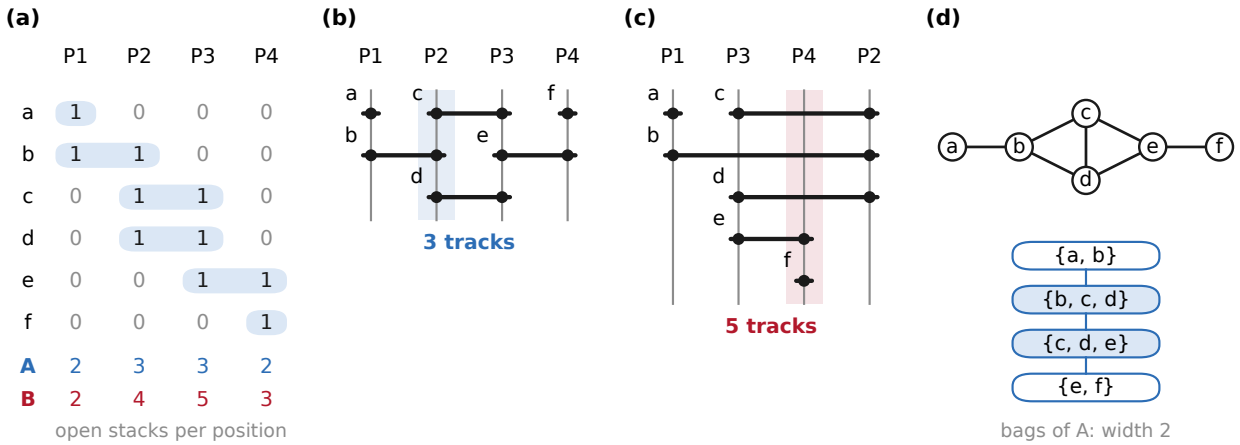


Figure 1.1: One instance, three readings, one number. (a) Six customers and four patterns, with each customer's open stack shaded in order A; order A keeps at most 3 stacks open and order B at most 5, and only 2 of the 24 orders reach 3. (b, c) The same matrix as a gate matrix layout: the same two orders need 3 and 5 tracks (the shaded gate is where the density peaks). (d) The same matrix as a *graph*, one clique per pattern: the open sets of order A are the bags of a path decomposition of width 2, so the optimum is the pathwidth plus one (Theorem 2.3).

1 Introduction

A factory makes a set of patterns, one at a time, and each customer's order needs some of them. A customer's stack opens when the first pattern the customer needs is made, and it closes when the last one is made. The *minimization of open stacks problem* (MOSP) asks for the order of the patterns that keeps the fewest stacks open at any one time. In the cutting industry, where the problem arose, the patterns are cutting patterns and the customers are piece types (Yanasse, 1997b). Figure 1.1(a) gives a small instance. Six customers, a to f, need four patterns: $P_1 = \{a, b\}$, $P_2 = \{b, c, d\}$, $P_3 = \{c, d, e\}$ and $P_4 = \{e, f\}$. Made in order A, $P_1P_2P_3P_4$, the patterns keep at most 3 stacks open. Made in order B, $P_1P_3P_4P_2$, they keep 5 open, because customer b is served first but waits for P_2, which comes last. Of the 24 orders, 2 reach 3 (order A and its reverse), 16 give 4 and 6 give 5.

The same matrix is a problem in VLSI design. In a *gate matrix layout* the patterns are gates, drawn as vertical lines in some order, and the customers are nets. Each net is a horizontal wire from the first gate it connects to the last, and wires that overlap must lie on different horizontal tracks. The task is to order the gates so that the fewest tracks suffice. Figure 1.1(b) and (c) draw the two orders. The wires are the same; only the gate order changes which wires overlap. Order A packs into 3 tracks and order B needs 5. The number of tracks a gate order needs is its *density*, the most wires crossing any one gate, and that is exactly the number of open stacks there. So the two problems compute the same number from the same matrix (Linhares and Yanasse, 2002, Proposition 2).

The same matrix is also a graph. Make each customer a vertex, and join two customers when some pattern needs both, so that each pattern becomes a clique (Figure 1.1(d)). Write down, for order A, the set of open stacks at each position: $\{a, b\}$, $\{b, c, d\}$, $\{c, d, e\}$, $\{e, f\}$. These sets cover every vertex and every edge, and each customer appears in a run of consecutive sets. In graph theory such a sequence is a *path decomposition*, its sets are *bags*, and its width is the size of

Table 1.1: Twelve problems from three fields, published as one family: eight rows are exact, counting the pathwidth row itself; two are bands; two are false as the table states them. Our own table of 2002, reproduced from Linhares and Yanasse (2002, Table 1, p. 1764) with its row order, wording and references as printed, and its caption: "Display of a family of equivalent problems studied independently on the literature". The shaded column corrects it: it gives the relation to pathwidth (pw) that is true, proved in Section 2 (Table 2.3). The last three rows, below the rule, are new: members of the family that the table missed, which bring a fourth field, numerical linear algebra.

Problem	Discipline	References	This paper
MOSP	Operations research	Yanasse (1997b); Fink and Voß (1999)	exact: pw + 1
Gate matrix layout	VLSI design	Möhring (1990); Wing et al. (1985)	exact: pw + 1
One-dimensional logic	VLSI design	Ohtsuki et al. (1979)	exact: pw + 1
PLA folding	VLSI design	Möhring (1990)	false[a]
Interval thickness	Graph theory	Kashiwabara and Fujisawa (1979)	exact: pw + 1
Node search game	Graph theory	Kirousis and Papadimitriou (1985)	exact: pw + 1
Edge search game	Graph theory	Kirousis and Papadimitriou (1986)	band: pw to pw + 2
Narrowness	Graph theory	Kornai and Tuza (1992)	exact: pw + 1
Split bandwidth	Graph theory	Fomin (1998)	band: pw to pw + 1
Graph path-width	Graph theory	Kinnersley (1992)	exact: pw (reference)
Edge separation	Graph theory	Lengauer (1981)	false[b]
Vertex separation	Graph theory	Kinnersley (1992)	exact: pw
Progressive pebbling[c]	Graph theory	Kirousis and Papadimitriou (1986)	exact: pw + 1
Vertex search[d]	Graph theory	Bienstock et al. (1991)	exact: pw + 1
Maximum wavefront[e]	Numerical linear algebra	Kumfert and Pothen (1997)	exact: pw + 1

New: not in Linhares and Yanasse (2002).

[a] Exact (pw + 1) for multiple folding, with any number of nets per track. [b] As cutwidth; the vertex separator game of the same reference is exact (pw, for graphs with an edge). [c] Added by us. Its relation to pathwidth is proved in two papers the table already cites, for other rows: Lengauer (1981, Thms 2–3) and Kirousis and Papadimitriou (1986, Thm 3.1). We prove it in Lean (Appendix B.3). [d] Added by us. Searchers occupy vertices and an invisible fugitive moves along unguarded paths. Exact on every graph with a vertex, edgeless ones included (Bienstock et al., 1991, (5.1)), and equal to node search when the graph has an edge. We prove it in Lean, following their construction for the upper bound (Appendix B.3). [e] Added by us. The least, over symmetric orderings of a sparse symmetric matrix with nonzero diagonal, of the largest wavefront (Kumfert and Pothen, 1997, §2.1); it is narrowness in another form. The source defines it but does not state the equality, which we prove in Lean (Appendix B.3).

its largest bag minus one. The least width over all path decompositions is the *pathwidth* of the graph, here 2. The construction works in both directions, so the fewest open stacks is always the pathwidth plus one (Theorem 2.3).

The same graph is, finally, a sparse symmetric matrix of numerical linear algebra, with a nonzero wherever two customers share a pattern. The frontal method eliminates its rows one at a time and must keep in memory the *wavefront*, the rows already touched but not yet eliminated. Eliminating the customers in the order $a, b, \ldots, f$ keeps at most 3 rows active at once, and no order does better: the least maximum wavefront is again the pathwidth plus one (Theorem 2.4). This fourth reading is not in our 2002 table, and we add it below.

We collected twelve such problems (Linhares and Yanasse, 2002), from operations research, VLSI design and graph theory, in the table that Table 1.1 reproduces. Each computes, on corresponding inputs, "a function $f(\Pi)$ that is either equal to the number of open stacks or closely

related to it (plus or minus one)" (Linhares and Yanasse, 2002, p. 1764). The table gives a reference for each problem, but no proofs and no graph of which problem reduces to which. "Plus or minus one" has two readings. Under the *literal* reading, f is within one of the number of open stacks on every input. Under the *fixed-offset* reading, f equals the number of open stacks plus one constant, the same on every input. Only the second makes solving one problem the same as solving another. We call the problems that are, on every input, pathwidth or pathwidth plus one the *pathwidth complex*: most of the table, and three problems we originally missed. Pathwidth is the quantity through which every true relation in the table is proved (Section 2). It is also the name the literature uses, more than three times as often as the other eleven names combined (Section 3), so a reader who searches under "open stacks" or "gate matrix layout" misses most of what is known. The communities that could have checked the table rarely read each other: of 844 works that cite the table's papers, only six cite both an open-stacks paper and a graph-theory paper.

The table is ours, and its history explains why no one checked which of its equalities are true. It was first printed in 2001, in the first author's doctoral thesis (Linhares, 2001, Table 2.1, p. 13), supervised by the second author, which proved MOSP NP-hard and equal to gate matrix layout. Its rows were collected over the years of the doctorate, while reading the literature of each field. Each came with a reference and nothing else: no row's relation to open stacks was checked, and no published proof was restated. The table appeared again, row for row, as Table 1 of Linhares and Yanasse (2002), and it has been read since as a statement of equivalence. Garcia de la Banda and Stuckey (2007), for instance, credit the speed of their search in this journal to "the fact that the problem is fixed parameter tractable (Linhares and Yanasse 2002)". When MOSP was set as the first Constraint Modelling Challenge in 2005, the organisers' report pointed to the table for the problems equivalent to it, "including graph path-width", and credited the second author with the idea of permuting customers rather than products (Smith and Gent, 2005; Yanasse, 1997b), the idea on which Chu and Stuckey's customer search (Section 4) branches. So the table is already used to justify algorithms, and whether it is true matters. If it is true, *every result* about any one of the twelve problems transfers to *all* the others. Looking back, the table was the thesis's most important finding, and this article does what the thesis did not: it states each problem precisely, proves what is true and refutes what is not. This paper asks: *which of the equalities in the Linhares–Yanasse table are true, and what follows when results, algorithms and benchmarks are moved between the twelve problems through pathwidth?*

Three more members. The table was also incomplete. Progressive pebbling, a game in which pebbles are placed on the vertices of a graph and each vertex is pebbled once, needs exactly pathwidth plus one pebbles (Kirousis and Papadimitriou, 1986, Thm 3.1), and Lengauer's black-white pebbling of a directed acyclic graph equals the pathwidth of its undirected version plus one (Lengauer, 1981, Thm 2). Both theorems are in papers that the table already cites, for its edge-search and edge-separation rows, yet pebbling itself is not among its problems. Bienstock et al. (1991, (5.1)) search for a fugitive who hides in the vertices, not the edges, and need exactly pathwidth plus one searchers on every graph with a vertex, including the edgeless graphs where the table's node search row fails. From numerical linear algebra, the wavefront of the frontal method, met above (Kumfert and Pothen, 1997, §2.1), is the problem of the table: the least maximum wavefront is the pathwidth of the matrix's graph plus one, on every nonempty matrix. We add the three as the last rows of Table 1.1, marked as new, and prove all three relations in

the Lean proof assistant (Appendix B.3). For pebbling and the vertex search, what is new is the membership: the equivalences were published, but the table did not include them. For the wavefront the equality itself is ours: it is elementary, as the wavefront is narrowness read in another form, but no source we have read states it.

What we find.

- *The table, settled.* Eight of the twelve rows are exact: on every input, outside degenerate cases such as an empty graph, the problem equals pathwidth or pathwidth plus one (the pathwidth row itself is one of the eight). Two rows, split bandwidth and edge search, lie in *bands*: within one and two of pathwidth, but not at a fixed offset. Two rows, PLA folding and edge separation, are false as stated, with counterexamples. A variant of each false row is exact. Three problems outside the table are exact members too: progressive pebbling, a vertex search, and the maximum wavefront of sparse matrix elimination (Section 2).

- *One name.* The literature has settled on "pathwidth", which has more than three times as many relevant works as the other eleven names combined. Since 2005, after pathwidth, the most active name of the twelve is MOSP, and the best exact search for the whole family came from that community, by way of the 2005 Constraint Modelling Challenge (Section 3).

- *Two false theorems, and our correction.* The best exact algorithm for MOSP, the *customer search* of Chu and Stuckey (2009), which we call the previous algorithm, branches on the order in which customers' stacks close, and it is also a pathwidth algorithm. We prove in Lean, however, that two of its published dominance theorems, the *definite move* and the *better move*, are false, as are their later restatements (Chu, 2011; Fink, 2012): each can discard the last remaining solution at a node of the search tree. We provide a counterexample instance: our implementation of their algorithm, with its better move off, obtains an optimum of 7, while our corrected version obtains 6, provable also by a SAT (Boolean satisfiability) solver. This is a theorem checked in Lean: the whole run of their algorithm on the instance, and its optimum of 6 (Theorem 4.9). Our corrected definite move was also found by Tamaki as a lemma (Kitsunai et al., 2016, Lemma 1), in the exact pathwidth literature; we show that it can be tested by one bipartite matching per candidate, and prove our corrected search sound. Our correction costs at most 2% more search nodes on each group of benchmarks we measured.

- *A pathwidth solver.* Ported to graphs, our corrected search is an exact pathwidth solver, which we release. Within 600 seconds per graph it proves the pathwidth of 11,194 of the 11,534 Rome graphs, a graph-drawing benchmark of 10 to 110 vertices; all 50 trees of VSPLIB, a benchmark library for vertex separation; and 39 of its 73 Harwell–Boeing graphs, the sparsity patterns of engineering matrices. On each set this is as many graphs or more than the published branch and bound of Coudert et al. (2016). A lower bound from graph minors, the pathwidth of a contracted copy of the graph (Theorem 4.11, proved in Lean), runs before each refutation: it proves the pathwidth of the Mycielski graph M_7 in a fraction of a second, 271 of the 340 Rome graphs left open at 600 s, and MOSP optima that took up to 1,128 core-hours to refute (Sections 4.7 and 5).

- *Benchmarks re-certified, and a dataset.* Our corrected search re-proved all 115 benchmark optima, on 113 distinct graphs, that rested on the previous algorithm, and none changed.

We release 17,714 graphs, distinct up to isomorphism, from twenty benchmark collections of MOSP, VLSI layout, vertex separation, graph drawing and treewidth; 16,089 have a certified pathwidth with a layout that attains it, and so a certified value for every problem of the pathwidth complex (Section 5).

How to read the paper. Section 2 states the twelve problems and proves what is true. Section 3 measures which names and communities the literature uses. Section 4 reads the exact open-stacks search as a pathwidth algorithm, finds the two false rules and proves our corrected search sound. Section 5 reports what our corrected search costs, how it performs as a pathwidth solver, and the dataset, and Section 6 returns to the question this paper asks. Every theorem we state is machine-checked in the Lean 4 proof assistant, unless the text says otherwise. The reader need not read Lean: each proof is given or cited in the text, and Appendix A maps each result to its formal statement. Appendix B holds the supporting material, and Appendix C the bibliometric method. The Lean development, both solvers, the dataset and an independent checker for each are openly available, the code and proofs as open source under the MIT licence (page 38).

2 The equivalences, proved

This section answers the first half of our question: which rows of the Linhares–Yanasse table are true, and in what sense. Section 1 separated two readings of "plus or minus one". We call a row *exact* when it meets the fixed-offset reading: its value is pathwidth, or pathwidth plus one, on every input. We call it a *band* when it meets only the literal reading: its value stays within a fixed distance of pathwidth, and both ends of that range occur. We call it *false* when it meets neither. The answer is that eight rows are exact, two are bands and two are false. Section 2.1 states the problems. Section 2.2 proves the answer in four steps: the hub theorem, which ties MOSP to pathwidth; the other exact rows; the bands; and the false rows. Table 2.3 and Figure 2.1 summarise it.

All theorems below are verified in the Lean proof assistant, unless the text says otherwise; Appendix A lists the formal statements. Where a result is published, we cite the published proof and do not repeat it.

Notation. Table 2.1 lists the symbols used in the rest of the paper. Graphs are finite and simple. A 0/1 matrix M has the customers as rows and the patterns as columns; in the VLSI problems the rows are nets and the columns gates. Row r is *active* at position j of a column order π if it has 1s in two columns x, y with $\pi(x) \le j \le \pi(y)$, possibly $x = y$: its stack is open there. "M has a 1" excludes only the all-zero matrix. One letter needs care. In this section k is the bound in each decision problem, so for pathwidth it bounds a width. In Section 4 k bounds the number of open stacks, which is one more than a width; we say so again there.

2.1 The problems

Each problem is a decision problem with an instance and an integer k; its value is the least k for which the answer is yes. Four of the twelve carry the rest of the paper, and we define them in full. Figure 1.1 shows each on one instance.

Table 2.1: Notation. Each symbol has one meaning throughout the paper; the symbols of the search are defined in Section 4.1.

Symbol	Meaning
$G = (V, E)$, $n = \|V\|$	a graph, its vertices and edges; in Section 4 the vertices are the customers, $V = C$
L	a *layout*, a bijection $V \to \{1, \ldots, n\}$
$X_1, \ldots, X_r$	the bags of a path decomposition
M; C; P	a 0/1 matrix; its rows (customers, nets); its columns (patterns, gates)
G_M	the *MOSP graph*: vertex set C, two customers adjacent when some pattern has a 1 in both rows; each pattern becomes a clique
H	the *net graph* of a circuit: two nets adjacent when they share a gate
π	an order of the columns
$Z(M)$, $t(M)$	the MOSP value (fewest open stacks) and the gate matrix layout value (fewest tracks)
pw, vs	pathwidth and vertex separation
θ, ns, mns, es, pes, ν, sb, cw, mcw, VSG, pla	the values of the other problems (Table 2.2)
k	the bound of a decision problem; in Section 4, a bound on open stacks, so width $k - 1$
w	a width, as in "pathwidth w"
$N[c]$	the closed neighbourhood of c: c and its neighbours
∂T, $b(T)$	the border of a vertex set T, the vertices outside T with a neighbour in T; $b(T) = \|\partial T\|$

Pathwidth $\mathrm{pw}(G)$**.** Is there a sequence of bags $X_1, \ldots, X_r \subseteq V$ covering V, with every edge inside some bag, $X_i \cap X_\ell \subseteq X_j$ whenever $i \le j \le \ell$, and $|X_i| \le k + 1$? (Kinnersley, 1992, p. 346). The condition on $X_i \cap X_\ell$ is the *interval property*: the bags that contain a vertex are consecutive.

Vertex separation $\mathrm{vs}(G)$**.** Is there a layout L such that for every $i < n$ at most k vertices u with $L(u) \le i$ have a neighbour v with $L(v) > i$? (Kinnersley, 1992, p. 346).

MOSP $Z(M)$**.** Given a 0/1 matrix M of customers by patterns, is there an order of the patterns with at most k customers active at every position? (Linhares and Yanasse, 2002, eqs. (1)–(2)).

Gate matrix layout $t(M)$**.** Given a net–gate matrix, is there an order of the gates and an assignment of nets to k tracks such that nets active at a common position get different tracks? (Möhring, 1990, p. 18); (Wing et al., 1985, Problem 1).

The other eight problems are needed only to read the table. Table 2.2 gives each in one line; Appendix B.1 gives the formal definitions, with the sources.

2.2 What is true

Table 2.3 gives the answer row by row, and Figure 2.1 draws it as a graph around pathwidth. The rest of this subsection proves it.

Table 2.2: The other eight problems of the Linhares–Yanasse table look unrelated (two circuit layouts, two search games and four graph problems), yet each asks for the least k that is possible. One line each. Formal definitions are in Appendix B.1.

Problem	In one line
One-dimensional logic	Order the gates of a circuit so that the intervals spanned by the nets overlap at most k at any point. Its graph is the *net graph* H, which joins two nets that share a gate. A boundary variant pins two gates to the ends.
PLA folding $\mathrm{pla}(M)$	Gate matrix layout in at most k tracks, with at most two nets on a track. With no limit per track (*multiple folding*) it is gate matrix layout itself.
Interval thickness $\theta(G)$	Add edges to G to make an interval graph whose largest clique has at most k vertices.
Node search $\mathrm{ns}(G)$	Searchers are placed on and removed from vertices. An edge is cleared when both its ends hold a searcher, and becomes contaminated again if a searcher-free path joins it to a contaminated edge. Clear every edge with at most k searchers at once. $\mathrm{mns}(G)$ is the same with no recontamination allowed.
Edge search $\mathrm{es}(G)$	Node search with a third move: sliding a searcher along an edge clears it. $\mathrm{pes}(G)$ is the same over *progressive* strategies, which never recontaminate an edge.
Narrowness $\nu(G)$	Add the vertices one at a time in some order, and drop each as soon as all its neighbours have been added. Never hold more than k.
Split bandwidth $\mathrm{sb}(G)$	Repeatedly split a vertex into an edge, sharing its neighbours between the two ends, so that the result has a layout in which adjacent vertices are at most k apart (its *bandwidth*).
Edge separation	The table's reference (Lengauer, 1981) supports three readings: cutwidth $\mathrm{cw}(G)$, the most edges crossing a gap of a layout; modified cutwidth $\mathrm{mcw}(G)$, the most edges passing strictly over a vertex; and Lengauer's vertex separator game $\mathrm{VSG}(G)$, a pebbling game that counts the unpebbled vertices adjacent to pebbled ones.

Table 2.3: The Linhares–Yanasse table, settled. Eight rows are exact, two are bands and two are false, and every true relation is a relation to pathwidth. *Relation*: what is true and proved in Lean, on the graph the problem lives on (the MOSP graph or the net graph H for the matrix problems, the input graph otherwise). *Published source*: the published work the row rests on; the Lean proofs are listed in Appendix A. *Status*: *exact*, a fixed offset; *band*, an inequality whose ends both occur; *false*, the table's claim fails, with a counterexample family proved in Lean.

Problem	Published source	Relation	Status
MOSP	Yanasse (1997a); Fellows and Langston (1989); Linhares and Yanasse (2002)	$Z(M) = \mathrm{pw}(G_M) + 1$ (M has a 1)	exact
Gate matrix layout	Möhring (1990)	$t(M) = Z(M) = \mathrm{pw}(G_M) + 1$	exact
One-dim. logic	Ohtsuki et al. (1979)	tracks $= \theta(H) = \mathrm{pw}(H) + 1$; boundary variant $\geq \mathrm{pw}(H) + 1$	exact
PLA folding	Möhring (1990)	$\max(\mathrm{pw} + 1, \lceil \lvert C \rvert / 2 \rceil) \leq$ pla, gap unbounded; multiple folding $= \mathrm{pw} + 1$	false; variant exact
Interval thickness	Möhring (1990); Kirousis and Papadimitriou (1985)	$\theta = \mathrm{pw} + 1$ ($V \neq \emptyset$)	exact
Node search	Kirousis and Papadimitriou (1985, 1986); Bienstock and Seymour (1991)	$\mathrm{ns} = \mathrm{mns} = \mathrm{vs} + 1 = \mathrm{pw} + 1 = \theta$ ($E \neq \emptyset$)	exact
Edge search	Ellis et al. (1994); Kirousis and Papadimitriou (1986)	$\mathrm{vs} \leq \mathrm{es} \leq \mathrm{vs} + 2$, also for pes	band
Narrowness	Kornai and Tuza (1992)	$\nu = \mathrm{pw} + 1$ ($V \neq \emptyset$)	exact
Split bandwidth	Fomin (1998)	$\mathrm{pw} \leq \mathrm{sb} \leq \mathrm{pw} + 1$	band
Pathwidth	Kinnersley (1992)	the reference	exact
Edge separation[a]	Lengauer (1981)	cw, mcw unbounded against pw (stars); $\mathrm{VSG} = \max(1, \mathrm{vs})$	false; VSG exact
Vertex separation	Kinnersley (1992)	$\mathrm{vs} = \mathrm{pw}$	exact
Not in the Linhares–Yanasse table			
Progressive pebbling	Lengauer (1981); Kirousis and Papadimitriou (1986)	$\mathrm{mpb} = \mathrm{pw} + 1$ ($V \neq \emptyset$), $= \mathrm{ns}$ ($E \neq \emptyset$)	exact
Vertex search	Bienstock et al. (1991)	$= \mathrm{pw} + 1$ ($V \neq \emptyset$), $= \mathrm{ns}$ ($E \neq \emptyset$)	exact
Maximum wavefront	Kumfert and Pothen (1997)	$\min \mathrm{wf} = \mathrm{pw} + 1 = \nu$ (matrix nonempty)	exact

[a] Also misattributed: what Lengauer (1981) proves exactly is the vertex separator game, which is the table's vertex separation row.

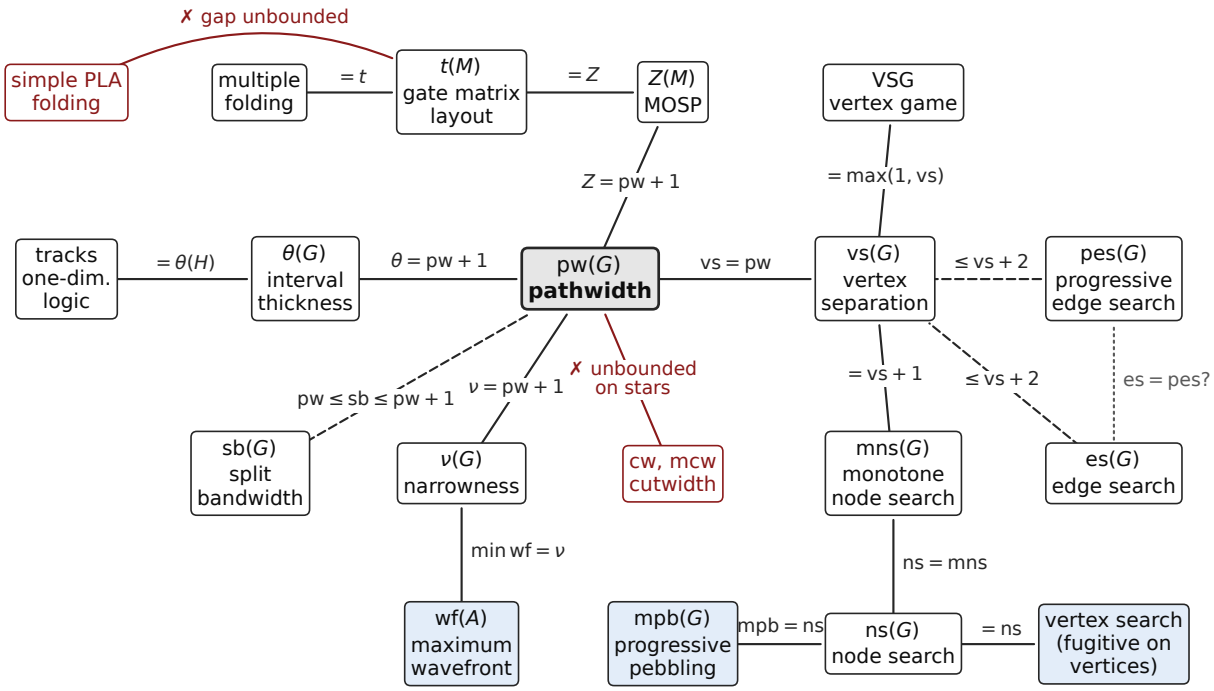


Figure 2.1: Pathwidth is the hub of the table: every exact relation is one or two steps from it, and the only failures are the two false claims, in dark red. Solid edges are exact relations and dashed edges bands, both proved in Lean; the dotted edge is LaPaugh's es $=$ pes, the one relation not proved, which no row needs. Each false claim is refuted by a counterexample family proved in Lean. The four matrix problems (top left) live on the MOSP graph G_M or the net graph H. Hypotheses are omitted: M has a 1, $V \neq \emptyset$, or G has an edge, as in Table 2.3. Two edges that follow from the others, mns $=\theta$ and ns $=$ vs $+\,1$, are not drawn. The shaded nodes on the bottom row are members the table missed. The vertex search edge holds whenever G has an edge, and the vertex search is pw $+\,1$ on every graph with a vertex (Bienstock et al., 1991, (5.1)) (Appendix A, A.61).

The hub. Two theorems tie the table to pathwidth. The first joins the two graph rows, vertex separation and pathwidth. The second joins MOSP to pathwidth, and through it all four matrix problems. The second rests on one lemma about path decompositions. For a vertex v, let $\text{first}(v)$ and $\text{last}(v)$ be the least and greatest indices of bags containing v; by the interval property, $v \in X_i$ exactly when $\text{first}(v) \le i \le \text{last}(v)$.

Lemma 2.1 (cliques sit in a bag)**.** *In a path decomposition of G, every clique of G lies inside one bag (Appendix A, A.1).*

Proof. The bags containing a vertex u of the clique form the interval $[\text{first}(u), \text{last}(u)]$. Two vertices of the clique are adjacent, so their edge lies in a bag and their intervals meet. Pairwise intersecting intervals of a line have a common point (the Helly property): let a be the largest left end; every right end is at least every left end, so at least a, and a lies in all of them. So all the intervals share an index i, and the clique lies in X_i. □

The first theorem is known. It says the two graph rows of the table are the same quantity.

Theorem 2.2 (vertex separation is pathwidth; known)**.** *For every graph,* $\text{vs}(G) = \text{pw}(G)$ *(Appendix A, A.2).*

This is Theorem 3.1 of Kinnersley (1992). Both directions are constructions. From a layout of separation k, each bag is the set counted at a cut together with the next vertex. From a decomposition, ordering the vertices by first gives a layout of the same width.

The second theorem is the one the rest of the paper uses. In Figure 1.1(d), the sets of customers active at each step of the good order are the bags of a path decomposition of the MOSP graph. The theorem says that this always works, in both directions.

Theorem 2.3 (MOSP is pathwidth plus one)**.** *If M has a 1, then* $Z(M) = \text{pw}(G_M) + 1$ *(Appendix A, A.3).*

Proof. ($\le$) Take a decomposition of G_M of width k. The rows of each column p form a clique, so by Lemma 2.1 they lie in a bag $X_{\beta(p)}$. Order the columns by $\beta(p)$. If row r is active at the position of column p, it has columns x, y with $\beta(x) \le \beta(p) \le \beta(y)$, and $r \in X_{\beta(x)} \cap X_{\beta(y)}$, so the interval property gives $r \in X_{\beta(p)}$. At most $k+1$ rows are active there.

($\ge$) Take an order with at most $Z = Z(M)$ active rows at every position. The active sets, with a singleton bag for each row with no 1, form a path decomposition: rows sharing a column are both active at its position, and a row is active on an interval of positions. Its width is $Z - 1$, using $Z \ge 1$. □

An independent, graph-theoretic route to the same equality is Theorem 2 of Suchan and Todinca (2007): every minimal interval completion of a graph with an edge clique cover is obtained by ordering the cover's cliques, and a MOSP matrix's patterns are such a cover of G_M. For the table, this settles the MOSP row as exact, with offset one, on the MOSP graph. Read as a cutting problem, Theorem 2.3 is Proposition 5 of Yanasse (1997a), which turns each pattern into a clique on its piece types. Read as a layout problem, it is Theorem 7 of Fellows and Langston (1989), with the column-expansion Lemma 4.1 of Fellows and Langston (1987). Linhares and Yanasse (2002, Prop. 2) identify MOSP with gate matrix layout. The proof above needs no assumption that the instance is reduced.

The graph matters. The MOSP literature also uses a second graph, with the patterns as vertices, joined when they share a customer, to split an instance into independent parts (Yanasse and Senne, 2010). It is natural to expect its pathwidth to measure the open stacks as well; it does not, in either direction. One pattern shared by four customers who each have a private pattern has $Z = 4$, while its pattern graph is the star $K_{1,4}$, of pathwidth 1 (Appendix A, A.4). A single customer with many patterns errs the other way.

The exact core. The other exact rows attach to Theorems 2.2 and 2.3 by published arguments. We do not reprove them on paper; each item below cites the published proof, and the Lean proofs are listed in Appendix A. We call the problems that equal pathwidth or pathwidth plus one on every input the *exact core*, the members of the pathwidth complex of Section 1: the eight exact rows, with multiple folding and the vertex separator game, and the three members outside the table (Appendix B.3), thirteen problems, or twelve if multiple folding is counted with gate matrix layout, which it equals.

Theorem 2.4 (the exact core)**.**

1. *Gate matrix layout is MOSP on the same matrix: if M has a 1, $t(M) = Z(M) = \mathrm{pw}(G_M) + 1$. For a fixed gate order the fewest tracks equals the most nets active at one position, by the left-edge algorithm (Möhring, 1990, p. 31) (Appendix A, A.5).*
2. *Interval thickness: if $V \neq \emptyset$, $\theta(G) = \mathrm{pw}(G) + 1$ (Möhring, 1990, Prop. 3.5) (Appendix A, A.6).*
3. *One-dimensional logic: if every net meets at least one gate, the fewest tracks is $\theta(H)$; if at least one net meets a gate (the matrix has a 1), it is $\mathrm{pw}(H) + 1$ (Ohtsuki et al., 1979, Thm 3) (Appendix A, A.7).*
4. *Narrowness: if $V \neq \emptyset$, $\nu(G) = \mathrm{pw}(G) + 1$, and for each order σ, $\nu(\sigma) = \mathrm{vs}(\sigma^{\mathrm{rev}}) + 1$ (Kornai and Tuza, 1992, Prop. 3.1) (Appendix A, A.8).*
5. *Node search: for every graph $\mathrm{ns}(G) = \mathrm{mns}(G)$, and if $E \neq \emptyset$, $\mathrm{ns}(G) = \mathrm{vs}(G) + 1 = \mathrm{pw}(G) + 1 = \theta(G)$ (Kirousis and Papadimitriou, 1985, Thm); (Kirousis and Papadimitriou, 1986, Thms 2.3, 4.1) (Appendix A, A.9).*
6. *Multiple folding: the fewest tracks is $t(M) = \mathrm{pw}(G_M) + 1$ (Möhring, 1990, Thm 3.14) (Appendix A, A.10).*
7. *Lengauer's vertex separator game: $\mathrm{VSG}(G) = \max(1, \mathrm{vs}(G))$ (Lengauer, 1981, Thm 4) (Appendix A, A.11).*
8. *Progressive pebbling: if $V \neq \emptyset$, $\mathrm{mpb}(G) = \mathrm{pw}(G) + 1$ (Kirousis and Papadimitriou, 1986, Thm 3.1) (Appendix A, A.60).*
9. *Maximum wavefront: for a symmetric matrix with nonzero diagonal and graph G, $V \neq \emptyset$, the least maximum wavefront is $\mathrm{pw}(G) + 1 = \nu(G)$, and for each order the maximum wavefront is vs of that order plus one (Appendix A, A.62).*
10. *Vertex search: if $V \neq \emptyset$, the least n admitting a successful search of Bienstock et al. (1991, §5) with every $|X_i| \leq n$ is $\mathrm{pw}(G) + 1$, whether or not the search is monotone, and it equals $\mathrm{ns}(G)$ when G has an edge (Bienstock et al., 1991, (5.1)) (Appendix A, A.61). The formal proof follows theirs for the upper bound; for the lower bound, where they use blockages, it reduces to node search.*

For the table, this makes the gate matrix layout, one-dimensional logic, interval thickness, node search and narrowness rows exact, and gives an exact variant of each false row. A single certified pathwidth therefore answers every problem in the exact core; Section 5.5 uses this.

Bands. Two rows are not fixed offsets of pathwidth, but stay within a fixed distance of it.

Theorem 2.5 (bands)**.** *For every graph,* $\mathrm{pw}(G) \le \mathrm{sb}(G) \le \mathrm{pw}(G)+1$ *(Fomin, 1998, Thm 8), and* $\mathrm{vs}(G) \le \mathrm{es}(G) \le \mathrm{vs}(G)+2$*, and the same for* pes *(Ellis et al., 1994, Thm 2.1) (Appendix A, A.12).*

Every end of both bands occurs. K_2 has sb $=$ pw $= 1$ and es $=$ vs $= 1$; the star $K_{1,3}$ has pw $= 1$ and sb $= 2$, and vs $= 1$ with es $= 2$; and $K_{3,3}$ has vs $= 3$ with es $= 5$. These values are computed by brute force from each quantity's definition, not proved in Lean.[1] Fomin (1998) assumes a connected graph with at least two vertices; the Lean proof needs neither. LaPaugh's equality es $=$ pes (LaPaugh, 1993) is the one relation we state and do not prove, and no row needs it.

For the table, this means the following. Relative to the number of open stacks $Z = \mathrm{pw}+1$, split bandwidth lies in $[Z-1, Z]$ and edge search in $[Z-1, Z+1]$. Both stay within one of Z, so they meet the literal reading of "plus or minus one". Since both ends occur, neither is a fixed offset, so neither meets the fixed-offset reading.

False rows. Two rows fail even the literal reading: no constant bounds their distance from pathwidth.

Theorem 2.6 (two rows are false)**.**

1. *PLA folding: for a matrix with a 1 and net set* C, $\max(\mathrm{pw}(G_M)+1, \lceil |C|/2 \rceil) \le \mathrm{pla}(M)$*, and the gap to* $\mathrm{pw}+1$ *is unbounded. The* 5×5 *identity needs 3 tracks while* $\mathrm{pw}(G_M)+1 = 1$*, and the path* P_n *as a matrix, one gate per edge, has* $t = 2$ *and* $\mathrm{pla} \ge \lceil n/2 \rceil$*, so the gap is unbounded on connected instances too (Möhring, 1990, Prop. 3.15) (Appendix A, A.13).*

2. *Edge separation read as cutwidth:* $\mathrm{pw} \le \mathrm{cw}$ *always, but* $\mathrm{cw}(K_{1,n}) \ge n/2$ *and* $\mathrm{mcw}(K_{1,n}) \ge n/2 - 1$ *while* $\mathrm{pw}(K_{1,n}) = 1$*; so neither cutwidth reading is within a constant of pathwidth (Appendix A, A.14).*

For the table, these rows are false as stated. Each has an exact variant (Theorem 2.4): the PLA claim holds for multiple folding, and the edge-separation reference proves exactly the vertex separator game.

Several published equalities need a nondegeneracy hypothesis, and one published proof has a gap that we correct; Appendix B.2 gives the details. Appendix B.3 adds three problems outside the table to the exact core.

Summary. Eight of the twelve rows of the Linhares–Yanasse table are exact, counting the pathwidth row itself, which is the reference. The other seven are pathwidth, or pathwidth plus one, exactly: MOSP, gate matrix layout, one-dimensional logic, interval thickness, node search, narrowness and vertex separation. So are two variants of the false rows, multiple folding and Lengauer's vertex separator game. Split bandwidth and edge search are bands of width one

[1] $\mathrm{sb}(K_{1,3}) \ge 2$ is also argued by hand: splits of a tree are trees and keep at least three leaves.

and two, within one of the number of open stacks but not at a fixed offset. Simple PLA folding and edge separation are false as stated. Three problems the table does not list are exact as well: progressive pebbling, the vertex search of Bienstock et al. (1991), and the maximum wavefront of Kumfert and Pothen (1997). One theorem is stated and not proved, LaPaugh's es = pes, and no row needs it. One of the table's references, Kashiwabara and Fujisawa (1979), is one we could not obtain; its row is proved from Möhring (1990) and Kirousis and Papadimitriou (1985).

3 Why *pathwidth*: the names and the communities

Section 2 showed that eight of the twelve rows of the Linhares–Yanasse table are one quantity. A result proved under any of their names therefore holds under all of them. But a reader who searches under one name will not find a result published under another. This section asks two questions about the literature. Which name does it use for the quantity? And, besides that name, which community has been most active? The answers are pathwidth, by far (Figure 3.1), and open stacks (Table 3.1). The second answer explains where the best exact algorithm for the family came from, and leads to Section 4.

How we counted. We searched OpenAlex (Priem et al., 2022) on 29 September 2026 for each problem's name and its common variants, in titles and abstracts, within Computer Science, Mathematics, Engineering and Decision Sciences. A search hit need not use the name in the table's sense: "narrowness" matches term rewriting, and "path-width" matches roads and tool paths. So we checked the 2,271 hits and kept the 1,593 that are about one of the twelve problems. The 700 hits for the eleven smaller names were judged one by one, by one annotator with model assistance; the 1,571 pathwidth hits were filtered by a stated rule, checked on samples. A mechanical phrase rule agrees with the judgements on 78% of the 700 and leads to the same conclusions. The counts are lower bounds, since a work can study a problem without naming it in its abstract. Appendix C gives the method in full, the robustness check and the counts per name. The census covers the twelve names of the table. The three members we add (Section 1) are not counted: progressive pebbling and the vertex search are graph-theory names, and the maximum wavefront belongs to a fourth discipline, numerical linear algebra.

Table 3.1: Besides pathwidth, the most active name over 2005–2024 is MOSP, although vertex separation led in 2015–2019. Relevant works per five-year period, for the three names with the most works over 2005–2024; the next are edge search (28) and node search (20).

Name	2005–09	2010–14	2015–19	2020–24	2005–24
Pathwidth	114	179	267	311	871
MOSP	17	17	5	12	51
Vertex separation	7	10	15	0	32

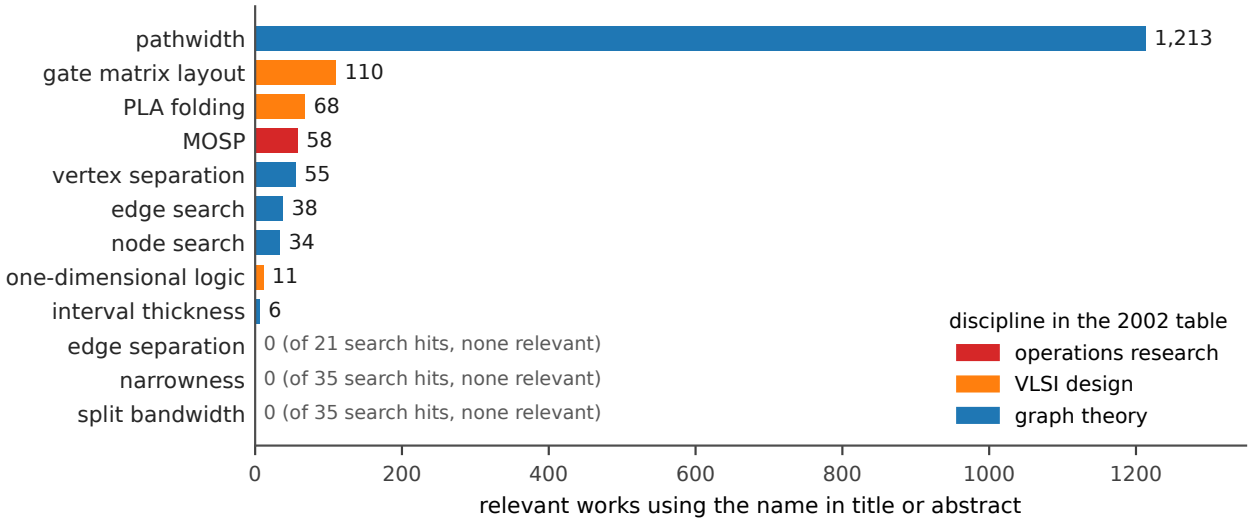


Figure 3.1: Pathwidth has more relevant works than the other eleven names combined, three times over, and three names have none. Works using each problem's name, in title or abstract, in the sense of the Linhares–Yanasse table; all years to September 2026. Colour gives the discipline that the table assigns the problem.

3.1 Pathwidth has absorbed the family

Figure 3.1 gives the counts. Pathwidth has 1,213 relevant works, 21 times as many as MOSP and more than three times as many as the other eleven names combined (380). Three names have no relevant work at all: narrowness, split bandwidth and edge separation. Two more are rare: interval thickness has 6 works and one-dimensional logic 11. Pathwidth is also the only name still growing: its 311 works in 2020–2024 are more than the whole history of any other name (Appendix C). So anything one would want to know about algorithms for this class of problems was most likely published under "pathwidth".

3.2 Besides pathwidth, open stacks

The VLSI names belong to the 1980s: since 2005 gate matrix layout has 9 relevant works and PLA folding 6. Table 3.1 shows the three names with the most works over 2005–2024. MOSP is second, with 51, ahead of vertex separation (32) and edge search (28). Vertex separation led only in 2015–2019. The open-stacks community, in operations research, is thus the one most active besides pathwidth.

Table 3.2: The open-stacks papers are read apart from graph theory. Works citing the papers that the Linhares–Yanasse table cites, by the discipline of the papers they cite. Of 135 works citing an operations-research paper, 6 also cite a graph-theory paper (4%); of 285 citing a VLSI paper, 65 do (23%). The 2002 paper itself counts as operations research. OpenAlex, 29 September 2026; 844 citing works in all.

		… of which also cite		
Works citing	All	operations research	VLSI	graph theory
operations research	135	–	14	6
VLSI	285	14	–	65
graph theory	507	6	65	–

It has also worked largely apart from graph theory. Table 3.2 counts the 844 works that cite the papers of the table, by the disciplines of the papers they cite. Only six works cite both an open-stacks paper and a graph-theory paper. Two are by the table's authors, one is open-stacks work on graph properties, and three are pathwidth or vertex-separation work from computer science. The VLSI papers are read with graph theory about five times as often.

3.3 From an open-stacks challenge to a pathwidth solver

It is not a coincidence that the best exact algorithm for MOSP came from the open-stacks community and is also, on our measurements (Section 5.3), among the strongest exact algorithms for pathwidth. The account is in Chu and Stuckey (2009, §1). MOSP was the subject of the first Constraint Modelling Challenge (Smith and Gent, 2005), posed in May 2005, which drew 13 entries. The fastest entry, by García de la Banda and Stuckey, combined dynamic programming with branch and bound and was an order of magnitude faster than any other (Garcia de la Banda and Stuckey, 2007). Chu and Stuckey (2009) built on it. They branched on which customer's stack closes next rather than on which pattern is made next, recorded the states already refuted, added dominance rules, and became five to six orders of magnitude faster on hard instances. Their solver closed every open instance of the challenge within ten seconds. MOSP's busiest decade in Table 3.1, 2005–2014, is the decade of this work.

The customer search reads an instance only through its MOSP graph (Chu and Stuckey, 2009, p. 4), so it decides pathwidth (Section 4). Ported to graphs, our corrected version of it proves more instances than reported on the benchmarks where published exact pathwidth results exist, those of Coudert et al. (2016). It proves the pathwidth of 11,194 of the 11,534 Rome graphs (97.1%), against their 95.6%, both within 600 seconds per graph. It proves all 50 VSPLIB trees, against their 30, and 39 of the 73 Harwell–Boeing graphs, against their 26. These are comparisons with published numbers obtained on other hardware, not a head-to-head run; Table 5.2 in Section 5 gives the full count.

Two consequences shape the rest of the paper. First, a result on any of the twelve problems should be stated, and searched for, as a result on pathwidth. Section 4 does this for the customer search. It finds that two of the search's published dominance theorems are false, and that the first, the definite move, is in its corrected form a lemma already known in the pathwidth literature. Second, a benchmark for any problem in the exact core is a pathwidth benchmark, which is how Section 5.5 builds its dataset.

4 An exact open-stacks search, read as a pathwidth algorithm

If MOSP is pathwidth (Theorem 2.3), then the best exact MOSP algorithm is a pathwidth algorithm, and its rules can be checked against what the pathwidth literature knows. This section does that for the *customer search* of Chu and Stuckey (2009), the search that Section 3 traced from the open-stacks challenge. It searches over the orders in which the customers' stacks close, not over pattern sequences, and it prunes with dominance rules. Three of the rules are stated as theorems (their Theorems 1, 2 and 3) and a fourth, the subset rule, in their §2. Chu and Stuckey (2009) place MOSP among "graph path-width and gate matrix layout", citing Linhares and Yanasse (2002), but they do not use pathwidth in the search. Any graph is the MOSP graph of the instance with one customer per vertex and one pattern per edge, so the search decides pathwidth for every graph, and ported to graphs, our corrected search is an exact pathwidth solver, which we release (Sections 3 and 5).

The stakes are these. Two of the published dominance theorems, the *definite move* and the *better move*, are false, and so are their restatements in Chu's thesis (Chu, 2011) and, with a stronger premise, in Fink (2012). The error rarely shows: on every benchmark the previous algorithm, the search as Chu and Stuckey (2009) publish it, answers correctly, and the only whole instance we found on which our implementation of it reports a wrong optimum, with the better move off, has 34 customers. There the *old move*, a rule that lets a failed sibling prune later branches, passes a false refutation on (Figure 4.2). And when our corrected search re-proved every benchmark optimum that rested on the previous algorithm, no value changed (Section 5). Our corrected definite move was also found by Tamaki, as the commitment lemma (Kitsunai et al., 2016, Lemma 1), which says when an exact pathwidth search may commit to one extension of a vertex set and try no other (Section 4.6). We show that it can be tested by one small bipartite matching per candidate. The main result, Theorem 4.8, says informally: *with both rules corrected, the search is sound; whenever it answers that k stacks do not suffice, they do not, and the graph has pathwidth at least k.* Our correction costs at most 2% more search nodes on each benchmark set (Section 5).

Section 4.1 defines the search and shows that it decides pathwidth, in both vocabularies. Section 4.2 gives the algorithm. Sections 4.3 and 4.4 take the rules one at a time, Section 4.5 proves our corrected search sound, and Section 4.6 places each rule in the pathwidth literature.

4.1 The search decides pathwidth

Chu and Stuckey (2009, p. 4) observe that the patterns matter only through the MOSP graph. So fix a graph $G = (C, E)$ whose vertices are the customers, and an integer $k \geq 0$. In this section k bounds the number of open stacks, which is one more than a width: k stacks suffice exactly when the pathwidth is at most $k - 1$ (Theorem 4.2). For a customer c write $N[c]$ for its closed neighbourhood, the stacks that must be open when c's stack closes. For a set $T \subseteq C$ of vertices, its *border* ∂T is the set of vertices outside T with a neighbour in T. For a set T of closed customers

write

$O(T) = \bigcup_{c \in T} N[c]$	the stacks opened once T has closed,
$\mathrm{fin}(X) = \{c : N[c] \subseteq X\}$	the customers free to close when X is open,
$\mathrm{cl}(T) = \mathrm{fin}(O(T))$	the *free closure* of T,
$b(T) = \lvert O(T) \setminus T \rvert = \lvert \partial T \rvert$	the stacks open after T has closed,
$o(c,T) = N[c] \setminus O(T)$, $\quad \mathrm{open}(c,T) = \lvert o(c,T) \rvert$	the stacks closing c would newly open.

The open stacks after T has closed are exactly the border of T, and this one identity carries the whole translation to pathwidth.

A *move* at T closes one customer $c \notin T$, at cost $\mathrm{cost}(T,c) = \lvert O(T \cup \{c\}) \setminus T \rvert = b(T \cup \{c\}) + 1$, the stacks open at that moment counting c. It is *playable* when $\mathrm{cost}(T,c) \leq k$. Write $\mathrm{Ext}_k(T)$ ("T extends") when some order of $C \setminus T$ has every step playable. On entering a node the search closes every free customer, so it stands only at *states*, sets with $S = \mathrm{cl}(S)$. The *child* of S by c is $S \cdot c = \mathrm{cl}(S \cup \{c\})$. The search's predicate $\mathrm{Sol}_k(S)$ holds when $S = C$, or when some playable c has $\mathrm{Sol}_k(S \cdot c)$ (Appendix A, A.15). Every node the search visits satisfies the invariant $b(S) \leq k$.

An example. Take the graph of Figure 1.1(d), with $k = 3$. At $T = \{a\}$ we have $O(T) = \{a, b\}$, so one stack, b, is open: $b(T) = 1$ and $\partial T = \{b\}$. No other customer is free, since $N[b]$ contains c and d, so T is a state. Closing b newly opens $o(b,T) = \{c, d\}$ and costs $\mathrm{cost}(T,b) = 3$, the stacks b, c and d; the move is playable. At $\{a, b, c\}$ every neighbour of d is open, so d is free, and the child $\{a, b\} \cdot c$ is $\{a, b, c, d\}$. The closing order a, b, c, e (with d and f closed free) costs $2, 3, 3, 2$, the profile of order A in Figure 1.1(a).

Lemma 4.1 (free moves never hurt)**.** *If $b(T) \leq k$, then $\mathrm{Ext}_k(T) \iff \mathrm{Sol}_k(\mathrm{cl}(T))$ (Appendix A, A.16).*

Closing a customer whose neighbourhood is already open opens nothing, never makes a later step dearer, and leaves one fewer stack open. The hypothesis is needed: closing the free customers one by one costs $b(T)$ at each step.

Theorem 4.2 (the search decides pathwidth)**.** *For nonempty C, $\mathrm{Sol}_k(\emptyset) \iff \mathrm{pw}(G) + 1 \leq k$. On the MOSP graph of an instance M that has a 1, $\mathrm{Sol}_k(\emptyset) \iff Z(M) \leq k$ (Appendix A, A.17).*

Proof. For a closing order $c_1, \ldots, c_n$ with prefixes S_i, the i-th step costs $1 + \lvert \partial S_i \rvert$. Reverse the order to get a layout. Its first $n - i$ vertices are $C \setminus S_i$, and those with a neighbour among the remaining ones are exactly ∂S_i. So the cost of the closing order is the vertex separation of the reversed layout plus one, and Lemma 4.1 at the root and Theorem 2.2 finish the proof. □

So a refutation, $\neg\mathrm{Sol}_k(\emptyset)$, may only mean $\mathrm{pw}(G) > k - 1$. The rest of the section asks whether the pruning rules preserve that meaning.

In pathwidth language. The exact pathwidth literature writes $N(T)$ for the border and $d(T)$ for its size (Kobayashi et al., 2014, p. 390); we keep ∂T and $b(T)$. A vertex sequence is *w-feasible* when every prefix has border at most w, and $\mathrm{vs}(G)$ is the least w for which some permutation is w-feasible. Table 4.1 translates each notion of the search into this vocabulary. Read it row by row: every ingredient of the customer search is a known ingredient of an exact pathwidth algorithm, and the last two rows are the rules of Section 4.2.

Table 4.1: The customer search is, ingredient by ingredient, an exact pathwidth algorithm: each notion of the search beside its name in the pathwidth literature. The memo and the definite move are described in Section 4.2.

Customer search	Layout
closing order from $\emptyset$	vertex sequence, read forwards
open stacks $b(T)$	border size $\lvert\partial T\rvert$
$\mathrm{cost}(T, c)$	$\lvert\partial(T \cup \{c\})\rvert + 1$
$\mathrm{Sol}_k(\emptyset)$	$\mathrm{vs}(G) \leq k - 1$
free closure $\mathrm{cl}(T)$	full set of T (Suchan and Villanger, 2009, p. 328)
memo	failure table
definite move, corrected	commitment to the child

Algorithm 4.1 The customer search (Chu and Stuckey, 2009) is a plain depth-first search; all its pruning sits in the memo (lines 4 and 12), the filter (line 6) and the old moves (line 9). It decides $\mathrm{Sol}_k(S)$ for a graph $G = (C, E)$ and a stack bound k, and is called as $\textsc{Search}(\emptyset, \emptyset)$. Q is the set of old moves the node inherits.

```
1  SEARCH(S, Q):
2    S ← cl(S)                                          free moves (Lemma 4.1)
3    if S = C then return true
4    if S is in the memo then return false              memo
5    A ← the c ∉ S ∪ Q with cost(S, c) ≤ k, in index order   cost cut
6    L ← filter of A: definite move, then subset rule, then better move   Sections 4.3–4.4
7    seen ← Q \ S
8    for each c ∈ L:
9      Q' ← {q ∈ seen : cost(S ∪ {q}, c) ≤ k}           old moves
10     if SEARCH(S ∪ {c}, Q') then return true
11     seen ← seen ∪ {c}
12   add S to the memo
13   return false
```

4.2 The algorithm

Algorithm 4.1 gives the customer search. It is a depth-first search over states. At each node it closes the free customers, consults a table of states already refuted, keeps the playable candidates, filters them with the dominance rules and recurses on the survivors.

Line 2 is safe by Lemma 4.1, and the cost cut of line 5 is exact, since an unplayable move is not a move of Sol_k. The rest is pruning, and each piece has a one-sentence intuition.

- *The memo* (lines 4 and 12): whether a state can be finished does not depend on how it was reached, so a state refuted once is refused when met again.
- *Old moves* (lines 7–11): a sibling whose subtree failed stays failed further down, as long as closing it first would have kept each later step within the bound.
- *The definite move*: if closing q now leaves no more stacks open than before, close q at once and try nothing else (Section 4.3 shows why this looks safe and is not).
- *The subset rule*: if d newly opens a subset of what r newly opens, then d is at least as good a move as r.

- *The better move*: if q would be a definite move right after r, then closing q before r is at least as good as closing r first.

Each of the three filtering rules discards a move r by citing a *covering*, or *link*, from r to a kept move c with $\mathrm{Sol}_k(S \cdot r) \Rightarrow \mathrm{Sol}_k(S \cdot c)$. The filter runs the definite move, then the subset rule, then the better move, each once. If the definite move fires, it keeps its customer alone.

This loop is the decision procedure of Kobayashi et al. (2014, Algorithm 1, p. 393), which asks whether a vertex set extends to a w-feasible permutation. It has four steps: replace the set by a committed extension, answer from a failure table or at the full vertex set, recurse on each feasible one-vertex extension, and record failures. The customer search is that loop with $w = k - 1$. The free closure and the definite move play the part of the commitment, the memo is the failure table, and the cost cut is the feasibility test. Coudert et al. (2014, §3.2) has the same three parts in a branch and bound. Neither line cites the other in the papers we have read: Chu and Stuckey (2009) predates the commitments and both 2014 papers, and none of the pathwidth papers cites it. This is the isolation measured in Table 3.2, at the level of one algorithm.

4.3 The definite move is false as published, and its correction is a matching test

The definite move is the strongest of the rules, since when it fires it discards every other candidate. Its premise looks like a safe accounting: closing q opens some new stacks and lets some customers close, and if it lets at least as many close as it opens, nothing seems lost by closing it first. Chu and Stuckey (2009, §3.1, p. 6) state:

> **Theorem 1.** Suppose S ++ [q] is playable and close(q, S) $\geq$ open(q, S), then if U′ = S ++ R is a solution, there exists a solution U = S ++ [q] ++ R′.

Here ++ is concatenation of closing sequences, R and R' are closing sequences, a *solution* is a complete closing order with every step playable, and $\mathrm{close}(q, S)$ counts the customers d with $o(d, S) \subseteq o(q, S)$ (p. 4). The rule keeps q alone. We read d as ranging over customers not yet closed, as our implementation does; read literally, closed customers count too, which weakens the premise, and both readings are refuted below. Chu (2011, Thm 6.3.6, p. 142) restates the theorem with the same premise and proof, reworded for k stacks. Write $X = S \cdot q$. The first lemma confirms that the premise says what the intuition says.

Lemma 4.3 (what the premise says)**.** $\mathrm{close}(q, S) = |X \setminus S|$. *Hence the premise holds iff* $b(X) \leq b(S)$*: the child has no more open stacks than the parent (Appendix A, A.24).*

The premise compares only the two ends, S and X. The rule fails because a set between them can be cheaper still. Figure 4.1(a) shows the graph on which it fails, and panel (b) the cheaper set between.

Counterexample 4.4. Let G be the graph of Figure 4.1, on customers $0, \ldots, 13$ with the 26 edges

$$0\text{–}3,\ 0\text{–}4,\ 0\text{–}7,\ 0\text{–}11,\ 1\text{–}2,\ 1\text{–}5,\ 1\text{–}6,\ 2\text{–}3,\ 2\text{–}4,\ 5\text{–}9,\ 5\text{–}11,\ 5\text{–}12,\ 6\text{–}8,$$
$$6\text{–}13,\ 7\text{–}9,\ 7\text{–}10,\ 7\text{–}13,\ 8\text{–}9,\ 8\text{–}10,\ 8\text{–}11,\ 9\text{–}10,\ 9\text{–}11,\ 10\text{–}11,\ 10\text{–}12,\ 10\text{–}13,\ 12\text{–}13,$$

with $S = \{2\}$, $k = 6$ and $q = 0$. Then $O(S) = \{1, 2, 3, 4\}$, $o(0, S) = \{0, 7, 11\}$ and $\mathrm{cost}(S, 0) = 6$, so 0 is playable. The customers with $o(d, S) \subseteq \{0, 7, 11\}$ are 0, 3 and 4, so $\mathrm{close}(0, S) = 3 \geq$

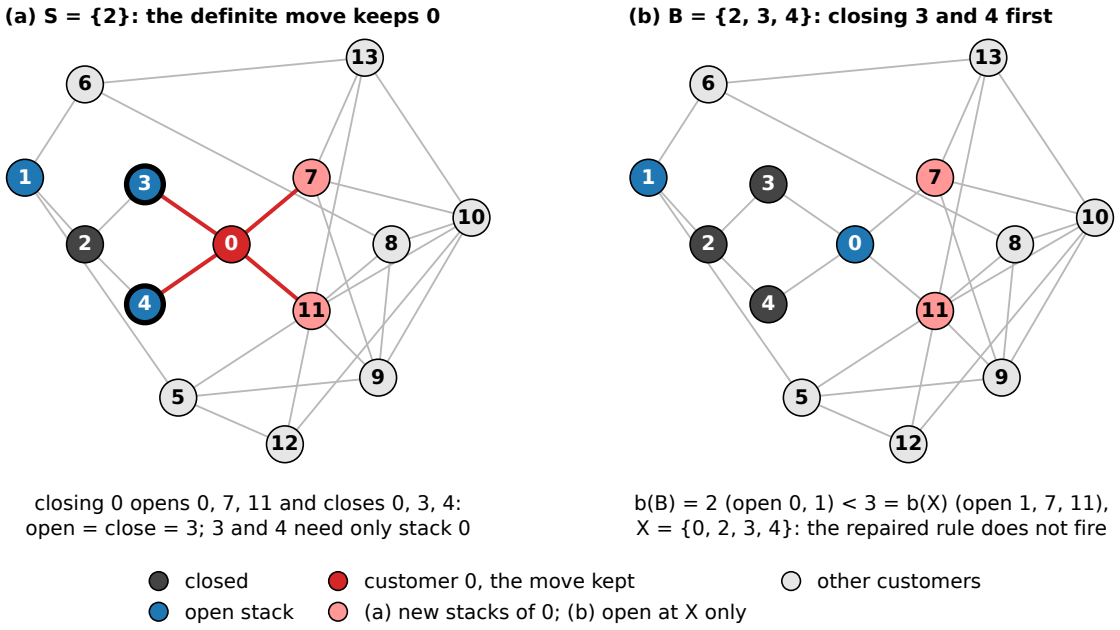


Figure 4.1: The definite move fails because a set between parent and child is cheaper than both. (a) The graph of Counterexample 4.4 at the state where the rule fires. Customer 2 is closed; 1, 3 and 4 are open. Moving 0 opens the new stacks 0, 7 and 11 and closes 0, 3 and 4, so open = close = 3 and the rule keeps 0 alone. But 3 and 4 have the single new stack 0 between them: closing them first gains two stacks for the price of one, which a solution starting with 0 cannot match. (b) The intermediate set $B = \{2, 3, 4\}$ leaves two stacks open, fewer than the three of the child $X = \{0, 2, 3, 4\}$, so the corrected rule of this section does not fire.

open$(0, S)$, and 0 comes first in index order, so the rule keeps only 0. A solution exists from S: the order

$$1, 3, 4, 6, 12, 13, 0, 5, 7, 8, 9, 10, 11$$

costs at most 6. None exists from the child $X = \{0, 2, 3, 4\}$: only seven sets are reachable from X by moves of cost at most 6, and none is C (Appendix A, A.25).

The published implication, as a universal statement, is refuted in Lean under both readings of close, and so is the thesis's Theorem 6.3.6 (Appendix A, A.26). Fink (2012, pp. 27–28) restates the rule with a stronger premise, counting only dominated customers of smaller index than q. The graph above refutes it under no labelling, but a third twin of 3 and 4 gives a 15-customer graph that does (Appendix A, A.27). The published proof moves q to the front of a solution and counts, after each prefix, the stacks q opens against the customers it lets close. But customers that the solution has already closed no longer count as closed by q, while the stacks q opens may still be new. Appendix B.4 gives the full analysis, and Appendix B.5 the search for smaller counterexamples.

The correction. We require the comparison at every set between the two ends, not only at S. Call q *hereditarily definite* at S when $b(X) \leq b(B)$ for every B with $S \subseteq B \subseteq X$ and $q \notin B$. By Lemma 4.3, the published premise is the case $B = S$. In the counterexample the hereditary

condition fails at $B = \{2,3,4\}$ (Figure 4.1(b)), where $b(B) = 2 < 3 = b(X)$ (Appendix A, A.28). The proof that the corrected rule is sound needs one inequality.

Lemma 4.5 (open stacks are submodular)**.** *For all $A, B \subseteq C$, $b(A \cup B) + b(A \cap B) \leq b(A) + b(B)$ (Appendix A, A.29).*

Proof. $b(T) = |O(T)| - |T|$, $O(A \cup B) = O(A) \cup O(B)$ and $O(A \cap B) \subseteq O(A) \cap O(B)$; the customers themselves balance exactly. □

Theorem 4.6 (the corrected definite move is sound)**.** *If $q \notin S$ is hereditarily definite at S, then $\mathrm{Ext}_k(S) \Rightarrow \mathrm{Ext}_k(X)$. At a state with the invariant, $\mathrm{Sol}_k(S) \Rightarrow \mathrm{Sol}_k(S \cdot q)$ (Appendix A, A.30).*

Proof. Take an order from S of cost at most k with prefixes T_i, in which q is the j-th step, and follow it from X, skipping customers already in X. A step $i < j$ closes c_i at $T_{i-1} \cup X$. We have $S \subseteq T_i \cap X \subseteq X$ and $q \notin T_i$, so the hypothesis gives $b(X) \leq b(T_i \cap X)$, and Lemma 4.5 gives $b(T_i \cup X) \leq b(T_i)$: the step costs no more than the original. After step $j-1$ the closed set contains T_j with the same opened set, and the rest of the order applies. The search's form follows by Lemma 4.1. □

Tamaki also found this theorem, as a lemma. The hereditary premise is the commitment condition of Tamaki, and the proof is the proof of the commitment lemma, Kitsunai et al. (2016, Lemma 1, p. 142); Section 4.6 makes the correspondence exact.

The matching test. The corrected rule seems to require checking exponentially many intermediate sets. It does not, and this cheap test is our algorithmic contribution in this section; we did not find it in the papers we hold. In words: the rule is safe exactly when all but one of the new stacks can be paired with a different freed customer who needed it. In the counterexample, closing 0 opens three new stacks and frees 3 and 4, both of whom needed only stack 0: one pairing where two are required.

Theorem 4.7 (the corrected premise is a matching condition)**.** *Let $Y = X \setminus (S \cup \{q\})$, and join each $d \in Y$ to the stacks of $o(d,S) \subseteq o(q,S)$. Then q is hereditarily definite at S iff this bipartite graph has a matching with at least* $\mathrm{open}(q,S) - 1$ *edges (Appendix A, A.31).*

Proof. The sets of the hereditary condition are $B = S \cup D$, $D \subseteq Y$, and $b(X) \leq b(S \cup D)$ iff $|D| - |o(D)| \leq \delta$ with $\delta = |Y| + 1 - \mathrm{open}(q,S)$, where $o(D)$ is the union of the $o(d,S)$. By the deficiency form of Hall's theorem the largest matching has size $|Y| - \max_D(|D| - |o(D)|)$, which is at least $|Y| - \delta = \mathrm{open}(q,S) - 1$ exactly when the hereditary condition holds. □

The bipartite graph has at most close -1 customers on one side and open stacks on the other, and the matching can stop at open -1 edges. Every q with $\mathrm{open}(q,S) \leq 1$ passes at once, so the definite move of the previous algorithm is right there (Appendix A, A.32).

4.4 The other rules

Each of the remaining rules is quoted, proved sound in the form used, and shown to need the test or tie-break it carries.

The subset rule. "If $o(c_i, S) \subseteq o(c_j, S)$ and $i < j$ ... move j can be removed" (Chu and Stuckey, 2009, §2, p. 5). As implemented, d dominates r when $o(d, S) \subseteq o(r, S)$ and the inclusion is strict or $d < r$. Every dominated playable r is dropped, with a fallback that keeps A if all would go. The covering is sound for every set S: if $o(d, S) \subseteq o(r, S)$ and r is playable, then d is playable and $\mathrm{Sol}_k(S \cdot r) \Rightarrow \mathrm{Sol}_k(S \cdot d)$, because d is free after r and closing d then r reaches the same state at no greater cost. With the index tie-break, domination is a strict order, so an undominated move with a solution survives, and the fallback never fires when a solution exists (Appendix A, A.33). The tie-break is necessary: without it, two customers with equal new stacks dominate each other and both go, which loses a solution on a 7-customer graph (Appendix A, A.34).

The better move. Chu and Stuckey (2009, §3.2, p. 6): "Suppose S ++ [q] and S ++ [r, q] are playable and close(q, S ∪ {r}) ≥ open(q, S ∪ {r}) then if U′ = S ++ [r] ++ R is a solution there exists a solution U = S ++ [q] ++ R′." Its premise is the definite move's premise for q at the child $\mathrm{cl}(S \cup \{r\})$, together with playability of r then q, and its conclusion swaps q and r (Appendix A, A.35). The swap is sound, because both orders end at the same state and neither step overflows. The appeal to the definite move is not.

Counterexample. On the graph of Counterexample 4.4 at the root with $k = 6$, the definite move does not fire, customers 0 and 2 both survive the subset rule, and the premise holds for $r = 2$, $q = 0$, so the rule drops 2 citing 0. But $\{2\}$ has a solution and $\{0\}$ has none (Appendix A, A.36). At that node another kept customer still has a solution, so this is a false link and not a lost node; no node where the better move loses the last solution has been found. Chu (2011, Thm 6.3.8, p. 143) states the better move with a different premise, $\mathrm{close}(q, S) \geq \mathrm{open}(q, S \cup \{r\})$, and proves it through his Theorem 6.3.6. The same graph refutes it at $S = \{2\}$, $r = 3$, $q = 0$ (Appendix A, A.37).

Correction and test. Cite q for r only if q is hereditarily definite at $\mathrm{cl}(S \cup \{r\})$, which is one matching at the child (Theorem 4.7); then the swap argument goes through (Appendix A, A.38).

Two requirements. The close count in the premise must not include the customers that r finishes on its own; counting them makes the rule's own conclusion false on a 12-customer graph. And the rules must run in the order definite, subset, better, each citing only customers still standing. If the better move runs first, the subset rule can cite a customer the better move has already dropped. An 8-customer graph then loses a node through a cycle of links that are each true (Appendix A, A.39). The second is an instance of the general observation of Beck et al. (2025, Prop. 3, Example 6) that mutually dominating transitions must be tie-broken.

The old move. Chu and Stuckey (2009, Thm 3, p. 7) let a child inherit a sibling q whose subtree failed, provided that inserting q before each later step keeps that step playable (line 9 of Algorithm 4.1). The rule is sound as implemented: if $\mathrm{cost}(S \cup \{q\}, c) \leq k$, then $\mathrm{Sol}_k(\mathrm{cl}(\{q\} \cup S \cdot c)) \Rightarrow \mathrm{Sol}_k(S \cdot q)$, so a refuted sibling stays refuted along any path whose steps pass the test (Appendix A, A.40). The test is necessary; the smallest graph that shows it is the path on five vertices (Appendix A, A.41).

The memo. Whether a state with $b(S) \leq k$ has a solution depends on the state alone, not on the path that reached it (Appendix A, A.42). One concern is that the memo and the old move might not combine: a refutation obtained with inherited old moves seems to depend on the ancestors, and our Python reference implementation of the search declines to run both for that reason. A

node taken on its own, with an old move that is not in fact refuted, can indeed record a solvable state (Appendix A, A.43). Theorem B.2 (Appendix B.7) shows that this never happens in a real run, because every old move is a refutation completed earlier in the same run.

4.5 Our corrected search is sound

What a rule must satisfy. At a state S with old moves Q, the cost cut keeps the playable candidates A and the filter keeps a sublist $L \subseteq A$. The filter is *node-sound* at (S, Q) if, whenever every $q \in Q$ has $\neg\mathrm{Sol}_k(S \cdot q)$, $\mathrm{Sol}_k(S)$ implies $\mathrm{Sol}_k(S \cdot c)$ for some $c \in L$ (Appendix A, A.44). In terms of coverings, a node is sound when two things hold. First, every link a rule cites is a true covering. Second, every chain of links from a discarded candidate ends in L or in Q. The second condition can fail through a cycle even when every link is true, which is why the order of the rules matters. Proposition B.1 shows that the corrected filter is node-sound, and Theorem B.2 that node soundness, the memo and the old move together give sound runs; both are in Appendix B.7. Together they give the main theorem.

Theorem 4.8 (our corrected search is sound)**.** *Run the customer search with free moves, the memo, old moves and the filter definite → subset → better. Let the definite move fire only on a hereditarily definite customer, and the better move cite only under the corrected premise. If the search refutes k from the root, then no closing order costs at most k,* $\mathrm{pw}(G) > k - 1$*, and on the MOSP graph of an instance M that has a 1,* $Z(M) > k$ *(Appendix A, A.45).*

In words: each rule throws away only moves that a kept move is proved to be at least as good as, and the memo and the old moves remember only failures that really happened earlier in the same search. So when the search says "no", no closing order fits in k stacks.

What is proved and what is tested. The theorem is about the search as specified; that our C and Python implementations run it is checked, not proved. Three checks link them:

- a transcription of the Lean filter agreed with the implemented filter on 16,244,090 node checks over graphs on 1 to 17 vertices;
- the matching agreed with the hereditary premise on 36,939,226 candidates;
- a search written from the theorems agreed with both implementations and an exact oracle on 17,431,232 runs over 52,592 graphs on 1 to 17 vertices.

Section 5.4 adds a certificate checker for individual refutations.

The previous algorithm. Under it, the filter is not node-sound (Appendix A, A.46). In a check of 570,206 runs with memo and old move, it lost the last solution at a node in 58 runs, all on graphs built around Counterexample 4.4 and all at k equal to the optimum. Every one found a solution through another branch and answered correctly.

A wrong optimum. A targeted search did produce a whole instance on which the previous algorithm (Chu and Stuckey, 2009) gives a wrong answer. It started from Counterexample 4.4, climbed toward graphs on which the previous algorithm refutes more and more of the root's

solvable branches, and joined two copies of a 17-customer near-miss by one edge. On the resulting 34-customer instance (Figure 4.2), we ran our implementation of the previous algorithm with its definite move, the subset rule and the old move, and with the better move off. It answers that 6 stacks do not suffice and that 7 do, so it reports an optimum of 7, with a refutation as its proof (Figure 4.2(a)). Yet the order of Figure 4.2(b) keeps at most 6 stacks open (6 tracks, in the figure's gate matrix reading); our corrected search finds it and refutes 5, so the optimum is 6. This is a theorem, checked in Lean.

Theorem 4.9 (the previous algorithm reports a wrong optimum)**.** *Let G_{34} be the MOSP graph of the 34-customer instance of Figure 4.2. Run with its definite move as published, the subset rule, the old move and the memo, and with the better move off, the previous algorithm refutes $k = 6$ on G_{34} and finds a solution at $k = 7$, so it reports an optimum of 7. The optimum is 6, and the pathwidth of G_{34} is 5 (Appendix A, A.47).*

The proof is a computation checked by the Lean kernel itself, with no unverified evaluation. A transcription of the previous algorithm is proved, for every graph and every k, to produce runs of the search of Section 4.1 with the published filter. On G_{34} it refutes 6 in 526 nodes with the memo and in 839 without it, the counts of our C and Python implementations, and without the memo it makes exactly the calls of our Python implementation, in the same order and at the same states. The optimum 6 is proved in Lean as well, by a sound filter. So the node-level failure of the published filter (Counterexample 4.4) is not only a possibility: on this instance it reaches the root, and the answer is wrong. What the theorem does not prove is that the transcription matches our code line for line; that rests on reading it and on tests, which every node count, the full trace and eight mutated traces passed. A SAT solver on our encoding of MOSP (Appendix D), which shares no code with either search, agrees: it finds an order with 6 open stacks and proves that 5 do not suffice, with a proof in DRAT, the standard proof format of SAT solvers, which the independent checker drat-trim verifies. The mechanism is the old move: the definite move falsely refutes a subtree, and the old move passes that refutation on to the later branches, which then skip the moves their solutions need. Without the old move, or with the better move switched on, the previous algorithm answers this instance correctly; whether it can also fail in that configuration is open. The failing configuration separates two rules that Chu and Stuckey run as one: "Our implementation of 'better move' subsumes 'definite move' so we will simply consider them as one improvement" (Chu and Stuckey, 2009, p. 6). Run in our implementation, their published configuration, with the better move on, answers this instance correctly. The theorems are false whatever the configuration, and the instance shows that a false dominance theorem can produce a wrong optimum. The instance, the order each search returns and a checker that needs nothing but Python are in the repository (`paper1/data/false_refutation/`, page 38).

4.6 The rules in pathwidth language

The definite move has an exact counterpart in the pathwidth literature, and this subsection makes the correspondence precise. Kitsunai et al. (2016, p. 142), stating Tamaki's notion, call an extension τ of a vertex sequence σ *committable* when $|\partial X'| \geq |\partial V(\tau)|$ for every X' with $V(\sigma) \subseteq X' \subseteq V(\tau)$. Their Lemma 1 says that if σ extends to a feasible permutation, so does τ.

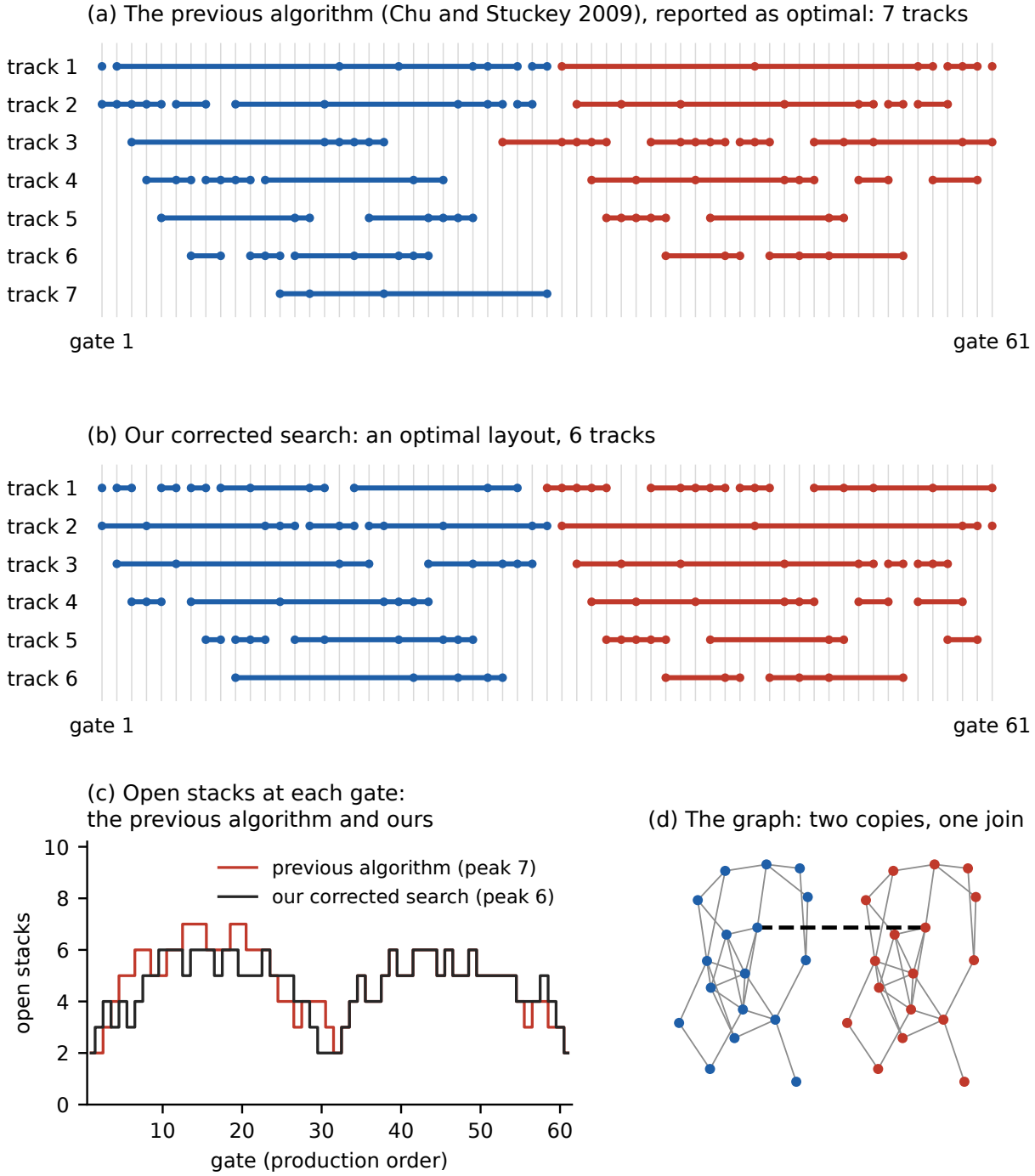


Figure 4.2: The previous algorithm (Chu and Stuckey, 2009) reports a wrong optimum. A 34-customer instance with 61 patterns, drawn as a gate matrix layout of 34 nets on 61 gates. (a) The layout the previous algorithm (our implementation, better move off) returns and certifies as optimal: 7 tracks. (b) A layout with 6 tracks, found by our corrected search, which also proves that 5 do not suffice, as do a SAT solver with a DRAT proof checked by drat-trim and Lean (Theorem 4.9). (c) The open stacks at each gate of the two orders: the previous algorithm's order reaches 7 in the first copy, where the optimal order never exceeds 6. (d) The instance's graph, two copies of a near-miss joined by one edge.

Lemma 4.10. *For $q \notin S$ and $X = \mathrm{cl}(S \cup \{q\})$: the published premise holds iff $b(X) \leq b(S)$, the commitment condition at the single set $X' = S$; and the corrected premise holds iff X is a commitment from S, the condition at every X' between them (Appendix A, A.49).*

So Theorem 4.6 is the commitment lemma at the child, and the published definite move is a commitment whose interior condition was dropped. In Counterexample 4.4 the endpoint test passes with $b(X) = 3 = b(S)$, while the interior set $\{2, 3, 4\}$ has border 2 (Appendix A, A.50). The published premise is not wasted: by Kitsunai et al. (2016, Lemma 10, Cor. 2, pp. 148–149) it guarantees a commitment to the least-border set between S and X, which need not contain q (Appendix A, A.51). In the counterexample that set is $\{2, 3, 4\}$ (Appendix A, A.52). Committing there when the matching fails is a second correction, which gives up no pruning; we have not built it.

A commitment of depth one, S to $S \cup \{q\}$, holds iff $\mathrm{open}(q, S) \leq 1$ (Appendix A, A.53). The greedy step of Coudert et al. (2014, Lemma 3, p. 49) is exactly that condition (Appendix A, A.54), and the push rule of Suchan and Villanger (2009, Rule 1, p. 330) is a case of it. So every published pathwidth rule of this kind lies in the region open ≤ 1, where the published definite move is right; Kobayashi et al. (2014, p. 388, Table 2) report depth-one commitments as "extremely effective". Chu and Stuckey's definite move reaches into open ≥ 2, and that is where it fails. For the cost of the test, Kitsunai et al. (2016, Cor. 2) find the least border between the ends of a given extension by a minimum s–t separator; Theorem 4.7 is that computation for the single target X, where it becomes a bipartite matching.

Table 4.2 sums up, and its rows are grouped by what they say. The published definite and better moves are false. The free move, the memo and the corrected definite move are known pathwidth results. The subset rule, the corrected better move and the old move have no counterpart we found, and are proved here, as is the matching test. The comparison is limited to the pathwidth papers we have read (Suchan and Villanger, 2009; Bodlaender et al., 2012; Coudert et al., 2014, 2016; Kobayashi et al., 2014; Kitsunai et al., 2016); "none found" is not a claim of priority.

Prior reports. We found no earlier report of the error in the 56 works we read in full of the 141 that cite the customer search's papers (Appendix B.6). The pathwidth literature has the correct form of the rule, as a commitment, and none of the pathwidth papers we have read mentions MOSP.

4.7 A lower bound from minors

The search spends nearly all its effort on one question: whether the value below the best known one can be refuted. Its floor, contraction degeneracy, is like every bound built from degrees and cliques a lower bound on treewidth plus one, so where pathwidth exceeds treewidth no such floor can meet the optimum: on the Mycielski graph `myciel6` (95 vertices, pathwidth 38) it is 25. The pathwidth side of the complex offers a bound with no such ceiling.

Theorem 4.11 (minor monotonicity)**.** *Let G be a graph and uv an edge of G, and let G/uv be the graph obtained by merging v into u: u becomes adjacent to every neighbour of u or v other than u and v, and v is removed. Then $\mathrm{pw}(G/uv) \leq \mathrm{pw}(G)$. Deleting a vertex or an edge never increases pathwidth either. Hence if a sequence of edge contractions and deletions turns G into H, then $\mathrm{pw}(H) \leq \mathrm{pw}(G)$ (Appendix A, A.48).*

Table 4.2: Two rules of the previous algorithm are false; the free move, the memo and the corrected definite move are known pathwidth results; the subset rule, the corrected better move and the old move, like the matching test, are proved here. Each rule of the customer search against the exact-pathwidth literature.

Rule	Published counterpart	Status
definite move, previous algorithm	endpoint case of a commitment (Lemma 4.10)	false (Cex. 4.4)
better move, previous algorithm	none found	false
free move	full sets (Suchan and Villanger, 2009); Kitsunai et al. (2016, Prop. 3)	known
definite move, open ≤ 1	depth-one commitments; greedy step; push rule	known
definite move, corrected	commitment lemma (Kitsunai et al., 2016, Lemma 1)	known; matching test (Thm 4.7) not found
memo	subset DP (Bodlaender et al., 2012); prefix and failure tables	known; with the old move, proved here (Thm B.2)
subset rule	none found	proved here
better move, corrected	none found	proved here
old move	none found	proved here

Proof. Take a path decomposition $X_1, \ldots, X_r$ of G of width $\mathrm{pw}(G)$, and replace v by u in every bag. No bag grows. Every edge of G/uv either is an edge of G not touching v, which keeps its bag, or joins u to a former neighbour x of v, whose bag held v and x and now holds u and x. The bags holding u were an interval of the path, and so were the bags holding v; since uv is an edge, some bag held both, so the two intervals meet and their union, the bags now holding u, is an interval. So the new bags form a path decomposition of G/uv of width at most $\mathrm{pw}(G)$. Deleting a vertex or an edge keeps the decomposition valid with every bag the same or smaller, and the last claim follows step by step. □

The theorem is the minor-closure of bounded pathwidth, stated for gate matrix layout by Fellows and Langston (1987, Lemma 4.2). It gives a certificate of a lower bound in two parts: a sequence of contractions turning G into a smaller graph H, which anyone can replay, and a refutation of $\mathrm{pw}(H) \leq w$, which then proves $\mathrm{pw}(G) \geq w + 1$; on the MOSP graph of an instance, by Theorem 2.3, it proves that every pattern order opens at least $w + 2$ stacks (Appendix A, A.48).

The minor step. Before our corrected search refutes the value below the best known one, w, on the whole graph, it tries to refute w on minors. It contracts the graph, merging a vertex of least degree into its neighbour of least degree (or, as a second rule, into the neighbour it shares fewest neighbours with), to sizes from half the vertices to 85% of them, smallest first, and on each minor it runs the same search under a node budget that triples from one size to the next. A refutation on any minor ends the descent with the value proved; otherwise the largest width refuted on a minor raises the lower bound, and the search on the whole graph proceeds as before. The cost of a refutation grows about tenfold for every five vertices, so the failed attempts on small minors

cost a fraction of the first success, and climbing from the smallest size costs about as much as knowing the right size in advance. The idea of solving a contracted graph exactly for a lower bound is used for treewidth (Bodlaender et al., 2006; Bodlaender and Koster, 2011); for pathwidth it is not capped by treewidth, which is where the rest of the floor stops. Section 5.3 reports what it settles.

5 Computational results and a dataset

Section 4 proved our corrected search sound. This section asks five practical questions about it and answers each with a measurement. What does our correction cost? At most 2% more search nodes on each group of benchmarks we measured. Do the published benchmark optima survive? All 115 that rested on the previous algorithm (Chu and Stuckey, 2009) were proved again, and none changed. How good a pathwidth solver is it? On every benchmark set with published exact counts it proves as many or more graphs than the published branch and bound of Coudert et al. (2016), and it is faster on each of the fourteen graphs whose times they publish. Can a reader check a refutation without trusting our code? Yes, up to about 75 customers, through a certificate and an independent checker. Finally, what does it make possible? A dataset of 17,714 graphs from twenty benchmark collections. For 16,089 of them the pathwidth is certified, and that gives the exact value of every problem in the exact core.

5.1 Experimental setup

Machine and code. Every run in this section used one machine, an AMD Ryzen 9 9950X (16 cores, 32 hardware threads) with 128 GB of memory under Ubuntu Linux. Both searches, the open-stacks customer search and its port to graphs, are written in C, compiled with gcc 15.2 using the flags `-O3` and `-march=native`, and driven from Python 3.12. A Python implementation of the same rules serves as a reference for cross-checks and emits the certificates of Section 5.4. Our corrected tests are the default in every implementation; the tests of the previous algorithm remain available as an option. The C implementation handles components of at most 1,024 vertices; 154 classes of the dataset of Section 5.5 are larger.

Runs. The basic job asks one question: is the optimum of this instance at most k? A *worker* is one single-threaded process that answers such questions, one at a time. A *node* is one state visited by the search, the unit in which we report effort. Up to 32 workers ran at once on the machine, so wall-clock times carry load noise. We therefore compare node counts and report time only in *core-hours*, the hours of one core summed over workers (32 workers for one hour make 32 core-hours). A *pair* runs the same question twice in the same worker, once under the previous algorithm and once under our corrected rules, alternating which goes first. A *descent* solves a whole graph: it lowers k from an upper bound until a refutation proves the last value optimal. Each table states its time or node budget.

Configurations. The open-stacks runs use two configurations of the rules of Section 4, which differ only in the better move. The *base* configuration leaves it off. The configuration *with better move* switches it on for instances whose customers need at most five patterns on average. This is

Table 5.1: Our correction never changed an answer and costs at most 2% more nodes in total. Each pair runs the same question under the previous algorithm and under the corrected rules in one worker. Totals are over the pairs that finished under both rules; a pair stopped by its budget says nothing about cost. *Vertices*: customers in the open-stacks rows (the vertices of G_M). *Largest pair ratio*: the largest ratio of corrected to previous nodes over single pairs. Open-stacks pairs refute the optimum minus one, at 600 s per call at 50–100 customers and 2×10^8 nodes at 125. The graph pairs run whole descents on VSPLIB, the colouring and named graphs, and a sample of 500 Rome graphs.

Instances	Vertices	Pairs finished	Nodes, previous	Nodes, corrected	Total ratio	Largest pair ratio
Open stacks: one refutation of the optimum minus one per pair						
MOSP corpus, base	9–40	6,135 of 6,135	228,147	228,154	1.00003	1.009
MOSP corpus, with better move	9–40	6,135 of 6,135	202,962	202,971	1.00004	1.040
Chu & Stuckey, with better move	50–100	118 of 125	1,358,096,671	1,362,205,210	1.00303	1.013
Chu & Stuckey, cap 2×10^8 nodes	125	11 of 23	90,969,508	90,961,419	0.99991	1.000
Graph pathwidth: one whole descent per pair, 120 s each						
VSPLIB, colouring, named, 500 Rome	4–2,916	731 of 880	2,977,849,807	3,038,670,717	1.020	–

the setting under which our implementation of the previous algorithm certified the benchmark values it proved.

Instances. The *MOSP corpus* is the 6,376 instance files of four public open-stacks collections, one of which is distributed twice. They are the 2005 Constraint Modelling Challenge (Smith and Gent, 2005), with a second copy of 46 of its files under other names, the random instances of Chu and Stuckey (2009) (30 to 125 customers), those of Faggioli and Bentivoglio (1998), and the 24 industrial instances of the SCOOP project, real instances from two sawmilling companies (Frinhani et al., 2018). They are the first rows of Table 5.4. A Chu–Stuckey name such as `Random-125-125-2-4_0` gives the number of customers, the number of patterns, the generator's density class (roughly, the number of customers per pattern) and the instance number. The graph benchmarks are those of Table 5.4.

Provenance. Every stored value comes with a witness, an order that attains it, and one of three provenances:

- *certified by search*: a search proved that no order does better, by refuting the value minus one;
- *certified by bound*: a lower bound equals the witness's value, so no search was needed;
- *upper bound only*: the witness is all there is, and optimality is not proved.

All 6,376 corpus values are certified (as of 12 October 2026; the last two were certified by the split search of Section 5.2).

5.2 The cost of our correction, and the benchmark values

Our corrected search adds one matching test to the definite move and to the better move (Section 4.3). A candidate that fails the matching is passed over, so the corrected tree can only differ where the matching fails. Table 5.1 shows how rarely that matters.

In every row of Table 5.1 our corrected search and the previous algorithm gave the same answer, and the total node counts differ by at most 2%. Single open-stacks pairs differ by at most

4%. The time per node also rises, by about 2% in the C implementation, measured on six hard instances at 75 to 125 customers. The published test passes and the matching then fails on only 0.07–0.27% of definite-move candidates, a rate that rises with size.

The benchmark values survive. None of the 115 values changed under our corrected rules. Of the 6,374 values certified before this work, 6,259 do not depend on the customer search at all. Each was proved in one of four ways:

- by an exhaustive computation over all subsets of customers (up to 15 customers);
- by a refutation from a SAT (Boolean satisfiability) solver whose proof an independent checker accepted, in DRAT, the standard proof format for SAT solvers;
- by an earlier SAT search;
- or by a lower bound equal to the value.

The other 115 rested on the customer search of the previous algorithm. Our corrected search refuted the value minus one on all 115, so all 115 values stand. They are 115 benchmark entries on 113 distinct graphs: SP3 and SP4 are each stored twice.

The seven hardest are 125-customer Chu–Stuckey instances at density classes 2 and 4. They are too large for one worker, so their search was split: the top levels of the tree are expanded into independent subtrees, each refuted on its own worker, and the value is refuted when every subtree is. The split is proved sound (Appendix A, A.55). All seven were refuted at 18 to 369 core-hours each, 1,112.6 in total (Table B.1, Appendix B.8).

Two values never certified before. The corpus also held two values that no method had certified: 25 for `Random-125-125-2-2_0` and 21 for `Random-125-125-2-3_0`. The same split refutes the value minus one on each. On 9 October 2026 it refuted 20 for `Random-125-125-2-3_0`, after 9,665 subtrees, 4.25×10^{12} nodes and 1,128 core-hours, which certifies 21 as its optimum. On 12 October 2026 it refuted 24 for `Random-125-125-2-2_0`, after 17,001 subtrees, 7.38×10^{12} nodes and 2,231 core-hours (Table B.1), which certifies 25; it is the largest refutation in this paper. With it, every value of the MOSP corpus is certified. The minor step of Section 4.7 reaches two of these values in seconds: a 69-customer minor of `Random-125-125-2-3_0` refutes 20 in 666,153 nodes, under a second of search, against 1,128 core-hours on the full instance, and an 87-customer minor of `Random-125-125-2-1_0` refutes 23 in 30 s with a larger budget, against 366.5 core-hours (Table B.1). It does not reach `Random-125-125-2-2_0`: every minor of up to 95 customers that we tried admits 24. The minor bound also equals the optimum on `Random-125-125-6-1_0` (81) and SP4 (53), which the search refutes on the full instance within a minute.

5.3 Our corrected search as a pathwidth solver

Ported to graphs, our corrected search is an exact pathwidth solver. Table 5.2 gives what it proves on the 11,915 benchmark graphs it was run on, counted as files, beside the run of the previous algorithm under the same caps and the published counts of Coudert et al. (2016). Where both runs prove a graph, the widths agree on every one. The 24 Rome graphs proved only by the

Table 5.2: Our corrected search as a pathwidth solver. Graphs whose pathwidth it proves, under our corrected rules and under the previous algorithm with the same caps (Rome: 10, 120 and 600 s in stages), without the minor step of Section 4.7, and the published counts of Coudert et al. (2016) under a 10-minute cap. *Width differs*: graphs proved by both runs with different widths.

Set	Vertices	Graphs	Proved, corrected	Proved, previous	Width differs	Coudert et al. (2016)
Rome	10–110	11,534	11,194	11,183	0	95.6% (ours 97.1%)
VSPLIB trees	22–202	50	50	50	0	30
VSPLIB Harwell–Boeing	24–960	73	39	39	0	26
VSPLIB grids	25–2,916	50	9	9	0	9 (sides 5–13)
TreewidthLIB colouring	5–864	58	31	31	0	14 of 14 agree; ours faster on all 14
freetdi named	4–3,282	150	125	125	0	–
All		11,915	11,448	11,437	0	

corrected run, and the 13 proved only by the previous one, sit near the 600 s cap. The comparison with Coudert et al. (2016) is with published numbers obtained on other hardware, not a head-to-head run. On the fourteen graph-colouring benchmarks of TreewidthLIB whose times Coudert et al. (2016, Table 4) report, our search proves the same pathwidth and is faster on every one: by 150 to 1,800 times on `david`, `miles750` and `mulsol.i.5`, by 70 to 80 times on `miles1000` and `zeroin.i.1`, and by 1.4 to 10 times on the other nine, where a hardware difference could account for the margin. It also proves `queen11_11` (pathwidth 87) in 37 s and `queen12_12` (103) in 391 s, beyond their table, and `fpsol2.i.1` (496 vertices, pathwidth 67) in 5.7 s, against 323 s for Kobayashi et al. (2014). Without the minor step, one graph goes the other way: the Mycielski graph M_7 (`myciel6`, 95 vertices, pathwidth 38), which their tuned branch and bound proves in about ten minutes, while the full refutation of 37 by our search does not finish in hours, its floor being 25. Mycielski graphs are built to have no triangles and yet to need many colours, so their largest cliques are single edges and their degrees are uneven: the degree and clique bounds that make up our floor see almost nothing. Their pathwidth also exceeds their treewidth (38 against 35 here), so no bound on treewidth can reach it.

With the minor step. The minor step of Section 4.7 changes this. A 58-vertex minor of `myciel6` already has pathwidth 38, and the step proves pw $=$ 38 in 0.2 s. On `myciel7` (191 vertices), which Coudert et al. (2016) leave open with pw $\leq$ 72, a 110-vertex minor proves pw $\geq$ 71; the best published lower bound we found is the treewidth bound 52 of Bodlaender et al. (2006), so pw is 71 or 72. Of the 340 Rome graphs left open by the 600 s run of Table 5.2, the step proves 271 within 60 s each (median 0.5 s); the Harwell–Boeing count rises from 39 to 43, and the colouring benchmarks gain `anna` and `myciel6`. On graphs proved both with and without the step, it lowers the total node count by 55 to 99% on Rome, the VSPLIB grids, Harwell–Boeing and the colouring graphs, adds 1% on the named graphs and 20% on the VSPLIB trees (about 1.5 s a tree), and costs 4% of the time on a sample of 120 MOSP corpus instances. The minor bounds for `myciel6`, `myciel7` and the four MOSP instances below were also refuted by a separate exact search that uses no dominance rule and writes out every refuted set, which an independent checker verifies. Their branch and bound is openly available in SageMath, and a head-to-head run on one machine is the comparison that would settle which solver is stronger; we have not yet made it.

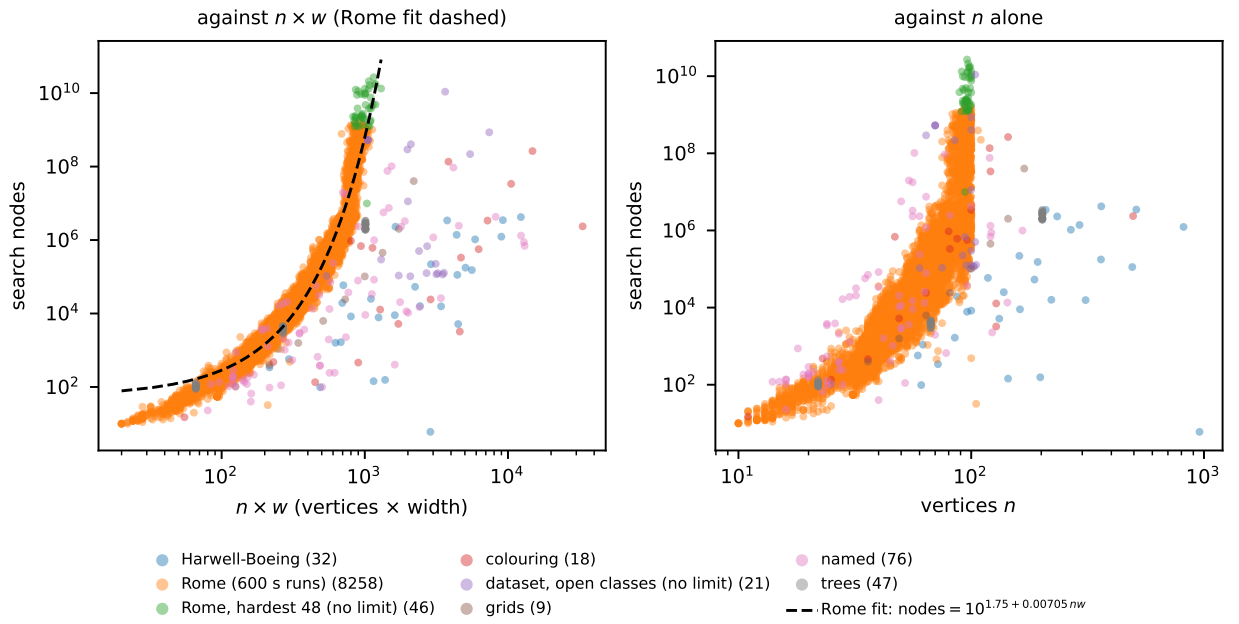


Figure 5.1: The cost of proving a pathwidth. Search nodes for every width our search has proved (8,507 graphs, both axes logarithmic), against n times the width (left) and against n alone (right). The dashed line is the fit on the Rome graphs, nodes $= 10^{1.75+0.00705\,nw}$. The structured benchmark sets fall far below it, and size alone gives no single law.

What a proof costs. Figure 5.1 plots the search nodes of every width our search has proved, on 8,507 graphs, against the number of vertices n and against n times the width w. On the Rome graphs, sparse graph-drawing instances, the cost follows one law,

$$\log_{10}(\text{nodes}) \approx 1.75 + 0.00705\,nw,$$

with $r^2 = 0.96$ and a typical graph within a factor of 2.3 of it: each further 100 in nw multiplies the work about fivefold. Size alone explains less ($r^2 = 0.85$), and so does width alone (0.89); the states of the search are vertex sets whose boundary has at most w vertices, and their number grows with both. The 48 hardest Rome graphs, which we refuted without a time limit (46 so far, the longest in 3.4 hours), lie above the line, so the law ranks graphs by hardness but understates the extreme tail. Graphs that the 600 s runs did not finish are absent, so the top of the curve is a lower envelope. The structured sets do not follow the Rome law: Harwell–Boeing graphs of up to 960 vertices, the colouring and named graphs and the dataset's open classes are proved with orders of magnitude fewer nodes than it predicts, mostly because a lower bound, often from a minor, meets the width early. Time tracks nodes closely ($r^2 = 0.98$), which is why we report nodes.

5.4 A checkable proof of a refutation

A refutation by the search is a claim about a tree of up to 10^{12} nodes, and Theorem 4.8 proves the rules sound, not the code that runs them. So the search emits a *certificate*: the tree it explored, every pruning step with its rule and its witness, and for each definite and better step the matching

Table 5.3: Every certificate the search emitted verifies, none is rejected, and checking takes milliseconds. Certificates for our corrected rules, configuration with better move, refuting the optimum minus one on the certified MOSP corpus at 9–75 customers, with 60 s to emit each. *Not emitted*: the emitter did not finish in time. Size is the compressed certificate; check time is the independent checker's.

Customers	Instances	Verified	Rejected	Not emitted	Median size (bytes, gzip)	Median check (ms)
9–10	1,614	1,614	0	0	332	0.04
11–20	2,508	2,508	0	0	356	0.08
21–30	1,816	1,816	0	0	414	0.24
31–40	197	197	0	0	650	1.27
41–50	120	120	0	0	12,045	33.24
51–60	1	1	0	0	126,508	278.86
61–75	30	19	0	11	26,516	143.70

of Theorem 4.7. An independent checker, which shares no code with the search, reads the instance matrix and the certificate. At every node it checks that

- the premise of every pruning step holds, recomputed from the matrix;
- every matching is valid: distinct customers, distinct stacks, each stack newly opened by its customer, and at least open $-$ 1 edges;
- every candidate within the bound is either a child or covered by a pruning step, and every chain of covers ends at a child or an old move, with no cycle;

and it replays the tree from the root. Each accepted step is a Lean theorem. That an acyclic covering by such steps is sound is a short paper argument, not yet a formal proof.

Table 5.3 gives the results. Every refutation emitted within its budget verifies, 6,275 of 6,286, and none is rejected. All 35.5 MB of certificates check in 147 s. The base configuration gives the same picture: 6,276 of 6,286 verify and none is rejected. The checker also catches the error of Section 4.3. On the graph of Counterexample 4.4, rooted at $\{2\}$, the previous algorithm emits a complete tree that claims there is no solution. A checker of the published premises accepts it. Ours rejects it, even with a largest matching attached, because the matching has one edge where two are needed. At 125 customers the trees are too large to ship as certificates.

5.5 A dataset with certified pathwidth

Section 2 showed that one certified pathwidth w answers every problem in the exact core. Pathwidth, vertex separation and the vertex separator game take the value w, and MOSP, gate matrix layout (which multiple folding equals), one-dimensional logic, interval thickness, narrowness, node search, progressive pebbling, the vertex search of Bienstock et al. (1991) and the maximum wavefront take $w + 1$: twelve problems in all, counting multiple folding with gate matrix layout. We therefore built one dataset from every benchmark collection of those problems that we could obtain, including the MOSP corpus.

The unit of the dataset is the *isomorphism class*: the set of instances whose graphs are the same up to renaming the vertices. For a matrix instance the graph is its MOSP graph. Two instances in one class have the same value under every problem, so the dataset holds one record per class, with its members and the renaming that maps each onto the record's graph. Classes were found

Table 5.4: Twenty collections, 21,754 instance files, reduce to 17,714 isomorphism classes, and 16,089 of them have a certified pathwidth. Largely open: PACE, the VSPLIB grids and Harwell–Boeing, and the MOSP sets of de Carvalho and Soma and of Frinhani et al.; Rome's 327 open classes are under 3% of it. *Classes*: isomorphism classes within the collection (78 graphs above 5,000 vertices are compared by file content only). *Counted here*: classes whose first member, in the order of the table, is in this collection, so the column sums to the dataset's class count. The next four columns split those classes by provenance; *No witnessed value*: no layout is stored yet. [a]The same 46 files distributed a second time under other names; every class is counted under its first copy.

				Certified			
Collection	Files	Classes	Counted here	by search	by bound	Upper bound only	No witnessed value
MOSP: Challenge 2005	5,806	3,169	3,169	3,168	1	0	0
MOSP: Challenge 2005 (second copy)[a]	46	46	0	0	0	0	0
MOSP: Chu & Stuckey	200	200	200	200	0	0	0
MOSP: Faggioli & Bentivoglio	300	287	274	274	0	0	0
MOSP: SCOOP	24	24	24	24	0	0	0
VLSI gate matrix circuits	11	11	11	5	5	1	0
VSPLIB trees	50	28	28	28	0	0	0
VSPLIB grids	50	50	50	9	0	19	22
VSPLIB Harwell–Boeing	73	72	72	32	7	33	0
Small (Marti et al. 2008)	84	84	84	32	52	0	0
Rome graphs	11,534	11,199	11,199	8,126	2,746	326	1
TreewidthLIB colouring	58	58	58	18	13	27	0
freetdi named graphs	150	146	143	97	22	23	1
freetdi control-flow graphs	1,817	1,070	1,064	398	661	4	1
MOSP: Frinhani et al. 2018, large	610	610	610	0	0	0	610
PACE 2016	291	288	112	37	18	51	6
PACE 2017 exact	200	199	193	61	7	115	10
PACE 2017 heuristic	200	197	173	11	3	45	114
PACE 2017 bonus	100	100	100	23	0	77	0
MOSP: Carvalho & Soma 2015	150	150	150	11	0	139	0
All	21,754		17,714	12,554	3,535	860	765

with nauty, a standard tool that computes a canonical form for each graph. Each record carries the graph, the width w, its provenance, and a witness layout of exactly that width. A record certified by bound also carries a *minor certificate*: a list of vertex deletions and edge contractions after which every vertex has degree at least w. Any checker can replay it, and it proves the pathwidth is at least w.

Table 5.4 gives the counts. The collections are the MOSP collections (Smith and Gent, 2005; Faggioli and Bentivoglio, 1998; Chu and Stuckey, 2009; de Carvalho and Soma, 2015; Frinhani et al., 2018), eleven VLSI gate matrix circuits (de Oliveira and Lorena, 2002), the VSPLIB sets (Duarte et al., 2012) and the Small graphs (Martí et al., 2008), the Rome graphs (Di Battista et al., 1997), and treewidth collections used as pathwidth benchmarks (van den Broek and Bodlaender, n.d.; Larisch and Salfelder, n.d.; Dell et al., 2017, 2018). Of the 17,714 classes, 16,089 carry a certified pathwidth with a witness layout and 860 a witnessed upper bound, and 765 have no witnessed value yet. The 610 large instances of Frinhani et al. (2018), with 400 to 1,000 customers, are included so that the dataset covers every published MOSP collection we have. None of them has a value yet, because an exact search at that size is out of reach.

An independent checker, sharing no code with the solvers, verifies the dataset. It re-reads all 20,987 member files with its own readers, maps each onto its record's graph by its renaming, recomputes the width of every witness layout and replays every minor certificate. It reports no failure on the 16,949 records, the 16,089 certified classes and the 860 with an upper bound. A value certified by search cannot be checked from its record; it rests on our corrected search, and

for instances within reach of the emitter the certificate of Section 5.4 is how a third party can check it.

Agreement with published values. All 84 Small graphs agree with the pathwidths of Mallach (2018, Tables 4–5). Ten of the eleven VLSI circuits are certified at their published best-known track counts (de Oliveira and Lorena, 2002, Table I); the eleventh, W4 with 202 nets, is open at 28 against 27.

A caution about MOSP files. A MOSP file can be read with rows as customers or with rows as patterns, and the two readings give different graphs. The same public repository ships the de Carvalho and Soma (2015) instances as customers by patterns and the Chu and Stuckey (2009) instances beside them as patterns by customers. Only published densities and values tell them apart. Read the wrong way round, an instance certifies at a different value.

6 Conclusion

We asked which equalities of the Linhares–Yanasse table are true, and what follows when results, algorithms and benchmarks are moved between its twelve problems through pathwidth. Eight rows are exact, counting the pathwidth row itself: each of those eight problems is pathwidth, or pathwidth plus one, on every input. Two rows, split bandwidth and edge search, are bands one and two wide, and two, PLA folding and edge separation, are false as stated, although a variant of each is exact (Table 2.3). This is the sense in which the exact problems form a pathwidth complex: they are one quantity, and the literature names that quantity pathwidth. Moving between the problems paid in both directions. The complex gets an excellent algorithm from the MOSP side and an excellent lower bound from pathwidth. Read as a pathwidth algorithm, the best exact open-stacks search turned out to rest on two false dominance theorems; our corrected definite move was also found by Tamaki as a lemma (Kitsunai et al., 2016, Lemma 1), and with it and our corrected better move the search is sound. Read from the other side, the minor-closure of pathwidth gives a lower bound that no degree or clique bound can reach, and it settles in seconds MOSP instances whose refutation took core-days (Section 4.7). And one certified pathwidth answers every problem of the pathwidth complex at once, which made a single dataset for all of them possible, including the three the table did not list.

What transfers, and what does not. Within the exact core, anything proved or computed for one problem holds for the others with the offset of Table 2.3: an algorithm, a lower bound, a benchmark value. Across a band only an approximation transfers. Split bandwidth and edge search stay within one of the number of open stacks, as the table's literal reading says, but not at a fixed offset, so an exact value of one does not give an exact value of the other. Across a false row nothing transfers. Simple PLA folding can need arbitrarily more tracks than the pathwidth suggests, and cutwidth is not within a constant of pathwidth on stars. Section 3 showed why these distinctions went unchecked for two decades: the communities rarely read one another, and most of what is known about the family is published under "pathwidth".

What the search teaches about dominance rules. The two false theorems were published in 2009 and restated in later work, and the implementations that use them reported no failure.

On every benchmark, and in every run of our checks where the previous algorithm (Chu and Stuckey, 2009) lost the last solution at a node, another branch found a solution (Section 4.5). It took a search built for the purpose to find a whole instance with a wrong optimum, 34 customers and one stack too many (Figure 4.2), in our implementation of the previous algorithm with the better move off; the optimum of 6 is confirmed by a SAT solver with a DRAT proof checked by drat-trim, and the whole run is a theorem checked in Lean (Theorem 4.9): a false dominance theorem, run as published, returns a wrong optimum. So the absence of wrong answers on benchmarks was never evidence that the rules were sound. Three findings of this paper bear on the soundness of dominance rules. Reading the rules in pathwidth language put the correct rule, the commitment lemma, beside the published one. Proving each rule on its own was not enough: the order in which the rules run matters, because a cycle of true links can still lose a node (Section 4.5). And a certificate whose checker recomputes every premise rejects the false refutation that a checker of the published premises accepts (Section 5.4). Our correction is cheap: one bipartite matching per candidate and at most 2% more nodes on each group of benchmarks we measured. Our corrected search re-proved all 115 benchmark optima that rested on the previous algorithm, and changed none.

What a reader can do differently. Cite our table as corrected here, Table 2.3, rather than the 2002 one. Look for results on MOSP and gate matrix layout under "pathwidth". Use our corrected search as an exact pathwidth solver. On every benchmark set with published exact pathwidth counts it proves as many or more graphs than the published branch and bound of Coudert et al. (2016) (Table 5.2). We believe it is currently the strongest exact pathwidth solver openly available, and it is open source. And in any implementation of the rules of Chu and Stuckey (2009), including the code behind the optima of Frinhani et al. (2018) that Martin et al. (2022) reuse (Appendix B.6), replace the definite and better moves of the previous algorithm with the matching test. The values those implementations report are not known to be wrong, but their refutations are not proved.

Limits. Six limits remain.

- The Lean model of the search and the C and Python code that run it are linked by testing and by certificates, not by proof (Section 4.5).
- The certificate checker of Section 5.4 accepts a refutation when every move the search discarded is justified by a checked step and the justifications never go round in a circle. That such a certificate really proves the refutation is argued on paper there, and is not yet formalised in Lean; it could be, in the style of our other proofs about the search.
- LaPaugh's equality es = pes is stated and not formalised. No row needs it (Section 2.2).
- One reference of the table, Kashiwabara and Fujisawa (1979), is one we could not obtain. Its row, interval thickness, is proved from two other references of the table (Section 2.2).
- The comparisons of the graph solver are with published numbers obtained on other hardware, not head-to-head runs, and only against Coudert et al. (2016). On one graph they report, the Mycielski graph M_7 (95 vertices, pathwidth 38), their tuned branch and bound proves the pathwidth in about ten minutes, and our search did not within 25 minutes.

- The census of names covers the twelve names of the table; the three members we add, and the wider treewidth and pebbling literatures, are not counted (Section 3).

Open problems. Every value of the MOSP corpus is now certified: the last two, `Random-125-125-2-3_0` and `Random-125-125-2-2_0`, fell on 9 and 12 October 2026 to the split search (Section 5.2), at 1,128 and 2,231 core-hours. Of the 17,714 classes of the dataset, 1,625 have no certified pathwidth. One more budget step for every open class within the size our implementation handles costs at most 500 core-hours, but on the Rome graphs each such step has closed only about 40% of what it attempted, so the graph sets will not close by budget alone. The 610 MOSP instances with 400 to 1,000 customers, and the 154 classes above the implementation's limit of 1,024 vertices per component (Section 5.1), are beyond an exact search. Beyond budgets, a class is certified without search only when a lower bound meets its witness (Section 5.5). The open classes are therefore also a test set for stronger lower bounds on pathwidth.

Data, code and proofs

Our code, proofs and data are openly available at `https://github.com/Alex-Linhares/MOSP_trough_SAT`; the code and the proofs are open source under the MIT licence. Everything this paper reports can be re-run from there, and most of it can be checked without re-running. The Lean proofs check in one build. Witnesses, minor certificates and the 34-customer instance have independent checkers, and refutations up to about 75 customers carry certificates (Section 5.4). The largest refutations can only be re-run, at up to 2,231 core-hours each (Section 5.2). The repository holds:

- the Lean 4 development, with every theorem the paper names (Appendix A) and an axiom check that covers all of them;
- the source of both our solvers, the open-stacks customer search and the graph pathwidth solver, in C and in Python, with our corrected rules as the default and the rules of the previous algorithm (Chu and Stuckey, 2009) as an option;
- the graph pathwidth solver as a standalone package (`pathwidth_solver/`), with the scripts that produced Table 5.2 and the published values it is compared with;
- the dataset of Section 5.5, the certificates of Section 5.4 and the independent checkers of both; the benchmark graphs keep the terms of their original distributions;
- the 34-customer instance of Figure 4.2, with the optimum our implementation of the previous algorithm reports with the better move off and its order, an optimal order, a checker that uses only the Python standard library, and the DRAT refutation of 5 stacks with the script that checks it (`paper1/data/false_refutation/`);
- the bibliometric data of Section 3: the cached OpenAlex responses, the relevance labels and the mechanical rule;
- a script for every table, figure and number in the paper, each of which regenerates it from the recorded data.

References

J. Christopher Beck, Ryo Kuroiwa, Jimmy H. M. Lee, Peter J. Stuckey, and Allen Z. Zhong. Transition dominance in domain-independent dynamic programming. In *31st International Conference on Principles and Practice of Constraint Programming (CP 2025)*, volume 340 of *LIPIcs*, pages 5:1–5:23, 2025. doi: 10.4230/LIPIcs.CP.2025.5.

Daniel Bienstock and Paul Seymour. Monotonicity in graph searching. *Journal of Algorithms*, 12 (2):239–245, 1991. doi: 10.1016/0196-6774(91)90003-H.

Daniel Bienstock, Neil Robertson, Paul Seymour, and Robin Thomas. Quickly excluding a forest. *Journal of Combinatorial Theory, Series B*, 52(2):274–283, 1991. doi: 10.1016/0095-8956(91)90068-U.

Hans L. Bodlaender and Arie M. C. A. Koster. Treewidth computations II. lower bounds. *Information and Computation*, 209(7):1103–1119, 2011. doi: 10.1016/j.ic.2011.04.003.

Hans L. Bodlaender, Thomas Wolle, and Arie M. C. A. Koster. Contraction and treewidth lower bounds. *Journal of Graph Algorithms and Applications*, 10(1):5–49, 2006.

Hans L. Bodlaender, Fedor V. Fomin, Arie M. C. A. Koster, Dieter Kratsch, and Dimitrios M. Thilikos. A note on exact algorithms for vertex ordering problems on graphs. *Theory of Computing Systems*, 50(3):420–432, 2012. doi: 10.1007/s00224-011-9312-0.

Geoffrey Chu and Peter J. Stuckey. Minimizing the maximum number of open stacks by customer search. In *Principles and Practice of Constraint Programming (CP 2009)*, volume 5732 of *Lecture Notes in Computer Science*, pages 242–257. Springer, 2009. doi: 10.1007/978-3-642-04244-7_21.

Geoffrey G. Chu. *Improving Combinatorial Optimization*. PhD thesis, Department of Computer Science and Software Engineering, The University of Melbourne, 2011. Minerva Access, http://hdl.handle.net/11343/36679.

David Coudert, Dorian Mazauric, and Nicolas Nisse. Experimental evaluation of a branch and bound algorithm for computing pathwidth. In *Experimental Algorithms (SEA 2014)*, volume 8504 of *Lecture Notes in Computer Science*, pages 46–58. Springer, 2014. doi: 10.1007/978-3-319-07959-2_5.

David Coudert, Dorian Mazauric, and Nicolas Nisse. Experimental evaluation of a branch-and-bound algorithm for computing pathwidth and directed pathwidth. *ACM Journal of Experimental Algorithmics*, 21:1–23, 2016. doi: 10.1145/2851494.

Marco Antonio Moreira de Carvalho and Nei Yoshihiro Soma. A breadth-first search applied to the minimization of the open stacks. *Journal of the Operational Research Society*, 66(6):936–946, 2015. doi: 10.1057/jors.2014.60.

A. C. M. de Oliveira and L. A. N. Lorena. A constructive genetic algorithm for gate matrix layout problems. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 21(8): 969–974, 2002. doi: 10.1109/TCAD.2002.800454.

Holger Dell, Thore Husfeldt, Bart M. P. Jansen, Petteri Kaski, Christian Komusiewicz, and Frances A. Rosamond. The first parameterized algorithms and computational experiments

challenge. In *11th International Symposium on Parameterized and Exact Computation (IPEC 2016)*, volume 63 of *LIPIcs*, pages 30:1–30:9, 2017. doi: 10.4230/LIPIcs.IPEC.2016.30.

Holger Dell, Christian Komusiewicz, Nimrod Talmon, and Mathias Weller. The PACE 2017 parameterized algorithms and computational experiments challenge: The second iteration. In *12th International Symposium on Parameterized and Exact Computation (IPEC 2017)*, volume 89 of *LIPIcs*, pages 30:1–30:12, 2018. doi: 10.4230/LIPIcs.IPEC.2017.30.

Giuseppe Di Battista, Ashim Garg, Giuseppe Liotta, Roberto Tamassia, Emanuele Tassinari, and Francesco Vargiu. An experimental comparison of four graph drawing algorithms. *Computational Geometry*, 7(5–6):303–325, 1997. doi: 10.1016/S0925-7721(96)00005-3.

Abraham Duarte, Laureano F. Escudero, Rafael Martí, Nenad Mladenovic, Juan José Pantrigo, and Jesús Sánchez-Oro. Variable neighborhood search for the vertex separation problem. *Computers & Operations Research*, 39(12):3247–3255, 2012. doi: 10.1016/j.cor.2012.04.017.

J. A. Ellis, I. H. Sudborough, and J. S. Turner. The vertex separation and search number of a graph. *Information and Computation*, 113(1):50–79, 1994. doi: 10.1006/inco.1994.1064.

Enrico Faggioli and Carlo Alberto Bentivoglio. Heuristic and exact methods for the cutting sequencing problem. *European Journal of Operational Research*, 110(3):564–575, 1998. doi: 10.1016/S0377-2217(97)00268-3.

Michael R. Fellows and Michael A. Langston. Nonconstructive advances in polynomial-time complexity. *Information Processing Letters*, 26(3):157–162, 1987. doi: 10.1016/0020-0190(87)90054-8.

Michael R. Fellows and Michael A. Langston. On search decision and the efficiency of polynomial-time algorithms. In *Proceedings of the Twenty-First Annual ACM Symposium on Theory of Computing (STOC '89)*, pages 501–512, 1989. doi: 10.1145/73007.73055.

Andreas Fink and Stefan Voß. Applications of modern heuristic search methods to pattern sequencing problems. *Computers & Operations Research*, 26(1):17–34, 1999. doi: 10.1016/S0305-0548(98)80001-4.

Claudia Fink. *O problema de minimização de pilhas abertas – novas contribuições*. PhD thesis, Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo, São Carlos, 2012.

Fedor V. Fomin. Helicopter search problems, bandwidth and pathwidth. *Discrete Applied Mathematics*, 85(1):59–70, 1998. doi: 10.1016/S0166-218X(97)00131-5.

Rafael de Magalhães Dias Frinhani, Marco Antonio Moreira de Carvalho, and Nei Yoshihiro Soma. A PageRank-based heuristic for the minimization of open stacks problem. *PLOS ONE*, 13(8):e0203076, 2018. doi: 10.1371/journal.pone.0203076.

Maria Garcia de la Banda and Peter J. Stuckey. Dynamic programming to minimize the maximum number of open stacks. *INFORMS Journal on Computing*, 19(4):607–617, 2007. doi: 10.1287/ijoc.1060.0205.

Toshinobu Kashiwabara and Toshio Fujisawa. NP-completeness of the problem of finding a minimum clique number interval graph containing a given graph as a subgraph. In *Proceedings of the 1979 IEEE International Symposium on Circuits and Systems*, pages 657–660, Tokyo, Japan, 1979.

Nancy G. Kinnersley. The vertex separation number of a graph equals its path-width. *Information Processing Letters*, 42(6):345–350, 1992. doi: 10.1016/0020-0190(92)90234-M.

Lefteris M. Kirousis and Christos H. Papadimitriou. Interval graphs and searching. *Discrete Mathematics*, 55(2):181–184, 1985. doi: 10.1016/0012-365X(85)90046-9.

Lefteris M. Kirousis and Christos H. Papadimitriou. Searching and pebbling. *Theoretical Computer Science*, 47:205–218, 1986. doi: 10.1016/0304-3975(86)90146-5.

Kenta Kitsunai, Yasuaki Kobayashi, Keita Komuro, Hisao Tamaki, and Toshihiro Tano. Computing directed pathwidth in $O(1.89^n)$ time. *Algorithmica*, 75(1):138–157, 2016. doi: 10.1007/s00453-015-0015-9.

Yasuaki Kobayashi, Keita Komuro, and Hisao Tamaki. Search space reduction through commitments in pathwidth computation: An experimental study. In *Experimental Algorithms (SEA 2014)*, volume 8504 of *Lecture Notes in Computer Science*, pages 388–399. Springer, 2014. doi: 10.1007/978-3-319-07959-2_33.

András Kornai and Zsolt Tuza. Narrowness, pathwidth, and their application in natural language processing. *Discrete Applied Mathematics*, 36(1):87–92, 1992. doi: 10.1016/0166-218X(92)90208-R.

Gary Kumfert and Alex Pothen. Two improved algorithms for envelope and wavefront reduction. *BIT Numerical Mathematics*, 37(3):559–590, 1997. doi: 10.1007/BF02510240.

Andrea S. LaPaugh. Recontamination does not help to search a graph. *Journal of the ACM*, 40(2): 224–245, 1993. doi: 10.1145/151261.151263.

L. Larisch and F. Salfelder. freetdi: named graphs and control-flow graphs. `https://github.com/freetdi/named-graphs`, `https://github.com/freetdi/CFGs`, n.d. Licence CC0 1.0.

Thomas Lengauer. Black-white pebbles and graph separation. *Acta Informatica*, 16(4):465–475, 1981. doi: 10.1007/BF00264496.

Alexandre Linhares. *Industrial Pattern Sequencing Problems: Some Complexity Results and New Local Search Models*. PhD thesis, National Institute of Space Research (INPE), São José dos Campos, Brazil, 2001. Doctoral thesis in Applied Computing, advisor H. H. Yanasse.

Alexandre Linhares and Horacio Hideki Yanasse. Connections between cutting-pattern sequencing, VLSI design, and flexible machines. *Computers & Operations Research*, 29(14):1759–1772, 2002. doi: 10.1016/S0305-0548(01)00054-5.

Sven Mallach. Linear ordering based MIP formulations for the vertex separation or pathwidth problem. *Journal of Discrete Algorithms*, 52–53:156–167, 2018. doi: 10.1016/j.jda.2018.11.012.

Rafael Martí, Vicente Campos, and Estefanía Piñana. A branch and bound algorithm for the matrix bandwidth minimization. *European Journal of Operational Research*, 186(2):513–528, 2008. doi: 10.1016/j.ejor.2007.02.004.

Mateus Martin, Horacio Hideki Yanasse, and Maria José Pinto. Mathematical models for the minimization of open stacks problem. *International Transactions in Operational Research*, 29(5): 2944–2967, 2022. doi: 10.1111/itor.13053.

Rolf H. Möhring. Graph problems related to gate matrix layout and PLA folding. In *Computational Graph Theory*, volume 7 of *Computing Supplementum*, pages 17–51. Springer-Verlag, Wien, 1990. doi: 10.1007/978-3-7091-9076-0_2.

Tatsuo Ohtsuki, Hajimu Mori, Ernest S. Kuh, Toshinobu Kashiwabara, and Toshio Fujisawa. One-dimensional logic gate assignment and interval graphs. *IEEE Transactions on Circuits and Systems*, 26(9):675–684, 1979. doi: 10.1109/TCS.1979.1084695.

Jason Priem, Heather Piwowar, and Richard Orr. OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts. arXiv:2205.01833, 2022.

Barbara M. Smith and Ian P. Gent. Constraint modelling challenge 2005. In *Fifth Workshop on Modelling and Solving Problems with Constraints, held at IJCAI 2005*, Edinburgh, 2005.

Karol Suchan and Ioan Todinca. Pathwidth of circular-arc graphs. In *Graph-Theoretic Concepts in Computer Science (WG 2007)*, volume 4769 of *Lecture Notes in Computer Science*, pages 258–269, 2007. doi: 10.1007/978-3-540-74839-7_25.

Karol Suchan and Yngve Villanger. Computing pathwidth faster than 2^n. In *Parameterized and Exact Computation (IWPEC 2009)*, volume 5917 of *Lecture Notes in Computer Science*, pages 324–335. Springer, 2009. doi: 10.1007/978-3-642-11269-0_27.

J.-W. van den Broek and Hans L. Bodlaender. TreewidthLIB. `http://www.cs.uu.nl/research/projects/treewidthlib/`, n.d. Graph files no longer served; colouring subset obtained from `https://mat.tepper.cmu.edu/COLOR/instances.html`.

Omar Wing, S. Huang, and R. Wang. Gate matrix layout. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 4(3):220–231, 1985. doi: 10.1109/TCAD.1985.1270118.

Horacio Hideki Yanasse. A transformation for solving a pattern sequencing problem in the wood cut industry. *Pesquisa Operacional*, 17(1):57–70, 1997a.

Horacio Hideki Yanasse. On a pattern sequencing problem to minimize the maximum number of open stacks. *European Journal of Operational Research*, 100(3):454–463, 1997b. doi: 10.1016/S0377-2217(97)84107-0.

Horacio Hideki Yanasse and Edson Luiz França Senne. The minimization of open stacks problem: A review of some properties and their use in pre-processing operations. *European Journal of Operational Research*, 203(3):559–567, 2010. doi: 10.1016/j.ejor.2009.09.017.

A Formal names

Every theorem, lemma and counterexample of this paper is checked in the Lean 4 proof assistant with the Mathlib library, unless the text says otherwise. The reader need not read Lean to follow the paper. This appendix is for a reader who wants to find the formal statement of a result: Table A.1 maps each result, in the order in which it appears, to the names of the Lean declarations that state it. Where a row cites a definition rather than a theorem, it is the definition the text uses. Lean names use `repaired` for what the text calls *corrected*.

All the theorems named depend only on the three standard axioms of Lean's classical logic, `propext`, `Classical.choice` and `Quot.sound`; an axiom check in the repository prints this for every theorem named in the paper. The development has one `sorry`, Lean's marker for a statement left unproved: a conjecture about lower bounds that no result of this paper uses. The part of the development behind Section 4 has no `sorry` at all.

Two Lean proofs take a route that the text does not describe. The proof of Theorem 4.2 goes through narrowness (Kornai and Tuza, 1992), which counts the closing order forwards rather than reversing it. The proof of Theorem 4.7 reduces the deficiency form of Hall's theorem to Mathlib's Hall theorem by adding δ dummy stacks. One Lean proof takes a different route from the published one: the proof that node search is monotone (Theorem 2.4(5)) follows the crusade argument of Bienstock and Seymour (1991), not the reduction to edge search of LaPaugh (1993) on which Kirousis and Papadimitriou (1986) rely. Three kinds of fact are computed rather than proved. The values of K_2, $K_{1,3}$ and $K_{3,3}$ after Theorem 2.5, which show that every end of the two bands occurs, are computed by brute force from each quantity's own definition. The search for smaller counterexamples (Appendix B.5) and the results of Section 5 are computations.

Table A.1: Every result of the paper has a formal statement in Lean, listed here in the order in which the results appear.

Entry	Result in the paper	Lean names
Section 2, the equivalences		
A.1	Lemma 2.1, cliques sit in a bag	`PathDecomposition.exists_bag_of_isClique`
A.2	Theorem 2.2, vertex separation is pathwidth	`vertexSeparation_eq_pathwidth`
A.3	Theorem 2.3, MOSP is pathwidth plus one	`MOSPInstance.mospValue_eq_pathwidth_add_one`
A.4	The pattern graph gives no bound: the star $K_{1,4}$ (after Theorem 2.3)	`star_mospValue`, `star_pathwidth_agreementGraph`, `star_refutes_pattern_graph_bound`
A.5	Theorem 2.4(1), gate matrix layout	`NetGateMatrix.tracks_eq_mospValue`, `NetGateMatrix.tracks_eq_pathwidth_add_one`
A.6	Theorem 2.4(2), interval thickness	`intervalThickness_eq_pathwidth_add_one`
A.7	Theorem 2.4(3), one-dimensional logic	`LogicArray.tracks_eq_intervalThickness`, `LogicArray.tracks_eq_pathwidth_add_one`
A.8	Theorem 2.4(4), narrowness	`narrowness_eq_pathwidth_add_one`

Entry	Result in the paper	Lean names
A.9	Theorem 2.4(5), node search	`nodeSearchMonotonicity`, `nodeSearch_eq_vertexSeparation_add_one`, `nodeSearch_chain`
A.10	Theorem 2.4(6), multiple folding	`NetGateMatrix.foldTracks_eq_pathwidth_add_one`
A.11	Theorem 2.4(7), Lengauer's vertex separator game	`vsg_eq_max`
A.12	Theorem 2.5, the bands	`pathwidth_le_splitBandwidth_le_pathwidth_add_one`, `vertexSeparation_le_edgeSearch_le_add_two`, `vertexSeparation_le_progressiveEdgeSearch_le_add_two`
A.13	Theorem 2.6(1), PLA folding	`plaTracks_idMatrix_five`, `plaTracks_idMatrix_unbounded`, `plaTracks_pathMatrix_unbounded`
A.14	Theorem 2.6(2), edge separation read as cutwidth	`cutwidth_unbounded`, `modCutwidth_unbounded`
Section 4, the search		
A.15	Step cost, Ext_k and Sol_k (Section 4.1)	`stepCost`, `Solvable`, `SearchSol`
A.16	Lemma 4.1, free moves never hurt	`solvable_iff_searchSol_cl`
A.17	Theorem 4.2, the search decides pathwidth	`searchSol_empty_iff_pathwidth_add_one_le`, `searchSol_mospGraph_iff_mospValue_le`
A.18	Table 4.1: closing order from $\emptyset$	`orderOfLayout`
A.19	Table 4.1: open stacks $b(T)$	`openStacks_eq_card_boundary`
A.20	Table 4.1: cost(T, c)	`stepCost_eq_card_boundary_add_one`
A.21	Table 4.1: $\mathrm{Sol}_k(\emptyset)$	`searchSol_empty_iff_pathwidth_add_one_le`
A.22	Table 4.1: free closure cl(T)	`mem_cl_iff`
A.23	Table 4.1: definite move, corrected	`isHereditarilyDefinite_iff_isCommittable`
A.24	Lemma 4.3, what the premise says	`closeCount_eq`, `isDefinite_iff`
A.25	Counterexample 4.4	`definiteMove_counterexample`, `cexFamily_closed`
A.26	Chu and Stuckey's Theorem 1 (the definite move), under both readings, and Chu's Theorem 6.3.6 are false	`chuStuckey_theorem1_false`, `chuStuckey_theorem1_false_literal`, `chuThesis_theorem636_false`
A.27	Fink's restatement of the definite move is false	`fink_theorem1_false`
A.28	The hereditarily definite premise, and its failure in Counterexample 4.4	`IsHereditarilyDefinite`, `not_isHereditarilyDefinite_cex`
A.29	Lemma 4.5, open stacks are submodular	`openStacks_submodular`
A.30	Theorem 4.6, the corrected definite move is sound	`solvable_cl_insert_of_hereditarilyDefinite`, `searchSol_cl_insert_of_hereditarilyDefinite`
A.31	Theorem 4.7, the corrected premise is a matching condition	`isHereditarilyDefinite_iff_hasDefiniteMatching`
A.32	The published definite move is right when open$(q, S) \leq 1$	`isHereditarilyDefinite_of_openCount_le_one`

Entry	Result in the paper	Lean names
A.33	The subset rule is sound	`searchSol_cl_insert_of_newlyOpened_subset`, `subsetFilter_sound`
A.34	The subset rule needs its tie-break (7-customer graph)	`noTieBreak_counterexample`
A.35	The better move's premise, restated	`isBetter_iff`
A.36	Chu and Stuckey's Theorem 2 (the better move) is false	`betterMove_counterexample`, `chuStuckey_theorem2_false`
A.37	Chu's Theorem 6.3.8 is false	`chuThesis_theorem638_false`, `chuThesis_theorem638_false_literal`
A.38	The corrected better move is sound	`IsRepairedBetter`, `searchSol_cl_insert_of_repairedBetter`
A.39	The better move's two requirements: the close count and the rule order	`bugA_counterexample`, `bugB_counterexample`
A.40	The old move is sound	`searchSol_reinsert`, `not_searchSol_of_oldMove`
A.41	The old move needs its reinsertion test (path on five vertices)	`reinsert_needs_test`
A.42	Solvability of a state does not depend on the path to it (the memo)	`solvable_iff_of_cl_eq`
A.43	A node with an unrefuted old move can record a solvable state	`exec_fake_oldMove`
A.44	Node-soundness of a filter (Section 4.5)	`NodeSoundAt`
A.45	Theorem 4.8, our corrected search is sound	`exec_repairedFullFilter_sound`, `exec_repairedFullFilter_pathwidth`, `exec_repairedFullFilter_mospValue`
A.46	Under the previous algorithm the filter is not node-sound	`not_codeFilterSound_cexGraph`
A.47	Figure 4.2: the previous algorithm refutes 6 and finds 7 on the 34-customer instance, whose optimum is 6 and pathwidth 5; its run is a run of the search with the published filter, which is therefore not sound there	`previousAlgorithm_wrong_optimum`, `not_filterSound_codeFilter`
A.48	Theorem 4.11: contracting an edge (merging v into u, as the solvers do) or deleting a vertex or an edge never increases pathwidth; along a legal contraction sequence, a minor of pathwidth above w forces pathwidth above w, and on the MOSP graph more than $w+1$ open stacks	`pathwidth_contractEdge_le`, `pathwidth_induce_le`, `pathwidth_mono`, `pathwidth_minorOf_le`, `pathwidth_gt_of_minorOf`, `MOSPInstance.lt_mospValue_of_minorOf`
A.49	Lemma 4.10, the premises as commitment conditions	`isDefinite_iff_endpoint`, `isHereditarilyDefinite_iff_isCommittable`
A.50	In Counterexample 4.4 the endpoint test passes and the commitment fails	`cex_isDefinite_not_isCommittable`
A.51	The published premise gives a commitment to the least-border set	`exists_isCommittable_of_isDefinite`
A.52	In Counterexample 4.4 that set is $\{2,3,4\}$	`cex_isCommittable_234`

Entry	Result in the paper	Lean names
A.53	A commitment of depth one holds iff open$(q, S) \leq 1$	`isCommittable_insert_iff`
A.54	The greedy step of Coudert, Mazauric and Nisse is that condition	`isGreedyStep_iff`
Section 5, computational results		
A.55	The split of the root into independent tasks is sound	`execSplit_repairedFullFilter_sound`, `execSplit_repairedFullFilter_mospValue`
Appendix B, supporting material		
A.56	Node search equalities need an edge (Appendix B.2)	`nodeSearch_ne_intervalThickness_of_edgeless`, `monotoneNodeSearch_ne_vertexSeparation_add_one_of_edgeless`
A.57	Lengauer's Theorem 4 fails at $k = 0$ (Appendix B.2)	`isPositiveVSG_triangleGraph_counterexample`
A.58	$t =$ ns fails on the matrix $[1]$ (Appendix B.2)	`tracks_ne_monotoneNodeSearch_one`
A.59	The Kirousis–Papadimitriou claim fails on $K_{1,3}$ (Appendix B.2)	`kirousisPapadimitriou_claim2_false`
A.60	Progressive pebbling is in the exact core (Appendix B.3)	`pbw_lengauerD_eq_pathwidth`, `pbw_eq_mospValue_pebbleMatrix`, `mpb_eq_pathwidth_add_one`
A.61	The vertex search is in the exact core (Appendix B.3)	`vertexSearchNumber_eq_pathwidth_add_one`, `monotoneVertexSearchNumber_eq_pathwidth_add_one`, `isNodeSearch_of_isSuccessfulVertexSearch`, `vertexSearchNumber_eq_nodeSearch`, `vertexSearchNumber_of_isEmpty`
A.62	The maximum wavefront is in the exact core (Appendix B.3)	`card_wavefront`, `maxWavefront_eq_vertexSepOfLayout_add_one`, `minMaxWavefront_eq_pathwidth_add_one`, `minMaxWavefront_eq_narrowness`, `minMaxWavefront_of_isEmpty`
A.63	Proposition B.1, the corrected filter is node-sound	`repairedFullFilter_sound`
A.64	Theorem B.2, runs are sound	`Exec.sound`
A.65	Appendix D, the SAT formulation is faithful (clauses 1–4)	`encodes_iff_mospValue_le`

B Supporting material

This appendix holds the material that supports Sections 2, 4 and 5 without being needed to follow them.

B.1 Formal definitions of eight problems

These are the formal definitions of the eight problems that Table 2.2 states in one line. As in Section 2.1, each is a decision problem with an integer k, and its value is the least k for which the

answer is yes.

One-dimensional logic. Given gates, nets and the nets each gate connects, is there an order of the gates whose net intervals have an interval graph of clique number at most k? Its *net graph* H joins two nets that share a gate. A *boundary* variant pins two gates to the ends (Ohtsuki et al., 1979, §§II–IV).

PLA folding $\mathrm{pla}(M)$**.** As gate matrix layout, with at most two nets per track. With no such bound (*multiple folding*) it is gate matrix layout itself (Möhring, 1990, pp. 25, 36).

Interval thickness $\theta(G)$**.** Is there an interval supergraph of G on the same vertices with clique number at most k? (Kirousis and Papadimitriou, 1985, p. 182); (Möhring, 1990, p. 31).

Node search $\mathrm{ns}(G)$**.** Can searchers, placed on and removed from vertices, clear every edge with at most k on the graph at once? An edge is cleared when both ends are guarded, and a clear edge is recontaminated by a searcher-free path to a contaminated one. $\mathrm{mns}(G)$ is the same over strategies that never recontaminate (Kirousis and Papadimitriou, 1985, p. 181); (Kirousis and Papadimitriou, 1986, §2).

Edge search $\mathrm{es}(G)$**.** The same game with a third move, sliding a searcher along an edge, which clears that edge; $\mathrm{pes}(G)$ for progressive strategies, which never recontaminate an edge (Kirousis and Papadimitriou, 1986, p. 208); (Ellis et al., 1994, p. 53).

Narrowness $\nu(G)$**.** Is there an order $v_1, \ldots, v_n$ such that the following process never holds more than k vertices: at step i add v_i, then remove every v_j, $j \le i$, with no neighbour v_ℓ, $\ell > i$? (Kornai and Tuza, 1992, §2).

Split bandwidth $\mathrm{sb}(G)$**.** Is there a graph obtained from G by repeatedly splitting a vertex into an edge, its neighbours partitioned between the two ends, with bandwidth at most k? (Fomin, 1998, §3.2).

Edge separation. The Linhares–Yanasse table cites Lengauer (1981), which supports three readings: cutwidth $\mathrm{cw}(G)$, the least over layouts of the most edges crossing a gap (p. 468); modified cutwidth $\mathrm{mcw}(G)$, counting only edges that pass strictly over a vertex (Definition 6); and Lengauer's vertex separator game $\mathrm{VSG}(G)$, the least k such that the vertices can be pebbled in an order that leaves at most k unpebbled vertices adjacent to pebbled ones (p. 467).

B.2 Edge cases and a gap in a published proof

Three of the published equalities fail on degenerate inputs, and each failure is proved in Lean. On a nonempty graph with no edges, $\mathrm{ns} = 0$ while $\mathrm{vs} + 1 = \theta = 1$, so both equalities of Kirousis and Papadimitriou (1985, 1986) need an edge (Appendix A, A.56). For a matrix with no 1s, $Z = 0$ while $\mathrm{pw} + 1 = 1$. Lengauer's Theorem 4 fails at $k = 0$ on an edgeless graph (Appendix A, A.57). Möhring (1990, Thm 3.9), $t = \mathrm{ns}$, fails literally on the 1×1 matrix $[1]$ (Appendix A, A.58).

The published proof of $\mathrm{vs} \le \mathrm{ns} - 1$ (Kirousis and Papadimitriou, 1986, Thm 4.1) has a gap. It orders the vertices by when they first receive a searcher and claims that every vertex counted at the worst cut still holds a searcher, "since the vertices in D_{i_0} are joined to vertices not yet cleared" (p. 217). A strategy may place a searcher and remove it again before clearing anything, which

recontaminates nothing. On $K_{1,3}$, placing and removing a searcher on each leaf before guarding the centre is an optimal recontamination-free strategy for which the claim fails (Appendix A, A.59). The theorem is true. The correction orders the vertices by clearing time, as our Lean proof does, or uses the paper's own normal form, its Corollary 2.4.

B.3 Members outside the table

Progressive black-white pebbling, which is not in the Linhares–Yanasse table, belongs to the exact core. Write pbw for its value and mpb for the minimum progressive pebbling number of Kirousis and Papadimitriou (1986). Lengauer (1981, Thm 3) gives $\mathrm{vs}(G) \le k \iff \mathrm{pbw}(G_d) \le k+2$ on the dag G_d built from G. His Theorem 2 gives $\mathrm{pbw}(D) = \mathrm{vs}(D_u) + 1$ on a dag D, where D_u is the MOSP graph of the matrix whose column v is v with its predecessors. And $\mathrm{mpb} = \mathrm{pw} + 1$ (Kirousis and Papadimitriou, 1986, Thm 3.1). All three are proved in Lean, each with its hypothesis: Lengauer's Theorem 2 in its instance form fails at $k = 1$ on a dag with no arcs, and $\mathrm{mpb} = \mathrm{ns}$ fails on edgeless graphs (Appendix A, A.60).

The vertex search. Bienstock et al. (1991, §5) define a search as a sequence of vertex sets $X_1 = \emptyset, \ldots, X_m$, each a subset or a superset of the one before; the fugitive may be at any vertex joined to its previous region by a path avoiding the current set, and the search succeeds when no such vertex remains. Their (5.1) states that a successful search with every $|X_i| \le n$ exists if and only if $\mathrm{pw}(G) \le n-1$, so the least n is $\mathrm{pw}(G)+1$ on every graph with a vertex; on an edgeless graph one searcher is needed, where node search needs none. We prove it in Lean (Appendix A, A.61), close to their proof. The upper bound is their construction: from a path decomposition $W_0, \ldots, W_L$ of least width, the search $\emptyset, W_0, W_0 \cap W_1, W_1, \ldots, W_L$ is monotone and successful, because after W_j, and again after $W_j \cap W_{j+1}$, the fugitive's region is exactly the set of vertices in no bag up to W_j. The lower bound reduces to node search: playing each change of position as single removals and then single placements turns a successful vertex search with at most n searchers into a successful node search with at most n searchers, since every contaminated edge keeps an endpoint in the fugitive's region. With $\mathrm{ns} = \mathrm{pw} + 1$ on graphs with an edge (Theorem 2.4), and $\mathrm{pw} = 0$ on edgeless graphs, this gives $\mathrm{pw}(G)+1$. A graph with no vertices needs no searcher. A brute force over all 1,252 graphs with at most seven vertices agrees. Where our proof departs from theirs is the lower bound: they derive it from blockages, their (2.1), a min-max theorem for pathwidth, while the formal proof replays the search as a node search and uses the monotonicity of node search proved for Table 1.1's node-search row.

The maximum wavefront. For a symmetric $n \times n$ matrix A with nonzero diagonal, Kumfert and Pothen (1997, §2.1) call row k active at step i if $k \ge i$ and $a_{kl} \ne 0$ for some $l \le i$; the ith wavefront is the set of active rows, and $\max_i |\mathrm{wf}_i(A)|$ is the storage the frontal method needs. In the graph G of A, $\mathrm{wf}_i = \{v_i\} \cup (N(V_i) \setminus V_i)$ with $V_i = \{v_1, \ldots, v_i\}$, so $|\mathrm{wf}_i|$ is one more than the number of later vertices with a neighbour among the first i. Hence for each order the maximum wavefront is the vertex separation of that order plus one, and the least over symmetric orderings is $\mathrm{vs}(G)+1 = \mathrm{pw}(G)+1$ when $n \ge 1$; an empty matrix gives 0 (Appendix A, A.62). The set wf_i is the shack of narrowness in its out-sequence form, so the least maximum wavefront is also $\nu(G)$. The source defines the wavefront but does not state the equality. A brute force over all 1,252 graphs with at most seven vertices and every ordering, 5.4 million pairs, agrees.

B.4 Why the published proof of the definite move fails

The proof moves q to the front of a solution and claims that after each prefix "we have at most open(q, S) extra stacks open, but at least close(q, S) extra stacks closed" (p. 6). At a prefix $T \supseteq S$ that does not yet contain q, moving q forward changes the open stacks by $b(T \cup X) - b(T) = |N[q] \setminus O(T)| - |X \setminus T|$. At $T = S$ this is open $-$ close ≤ 0. But customers of X that the solution has *already* closed are in T and no longer count as extra stacks closed, while the stacks q opens may still be new. In the example, a solution may begin with 3, after which 4 is free: at $T = \{2, 3, 4\}$, moving 0 forward costs one more stack. A second, independent slip: even where the conclusion holds, the literal sequence $S +\!\!+ [q] +\!\!+ R'$ need not be a solution, because a customer that q frees stays open until its turn; the argument needs the free moves.

B.5 The search for smaller counterexamples

The smallest graph on which the rule loses the last solution has 14 customers: there is none on any graph to 9 vertices, by exhaustion, and none on the 88,802 connected graphs on 10 vertices with at most 14 edges. The counterexample was found by a search over a family of gadgets built from the failed proof, and minimised by deletion.

B.6 The search for prior reports of the error

We looked for any earlier report of the error in the definite move. We listed the works citing Chu and Stuckey (2009), Chu's thesis and its IJCAI 2013 abstract in OpenAlex, Semantic Scholar and OpenCitations: 141 works. Of the 38 that cite the CP paper we read 24 in full (63%), and of the 103 that cite only the thesis, 32 (31%). No work read gives a counterexample, an erratum or a correction. One, Fink (2012), restates the definite move as true with a stronger premise, refuted in Section 4.3. Three use an implementation of the rules and report no failure; among them Frinhani et al. (2018) ran the original code to obtain the optima that Martin et al. (2022) reuse.

B.7 The filter and the runs are sound

Proposition B.1. *At every state with the invariant, for every set Q of refuted old moves, the filter definite $\rightarrow$ subset $\rightarrow$ better with both corrections is node-sound (Appendix A, A.63).*

Proof. If the definite move fires, Theorem 4.6 applies. Otherwise the subset rule keeps a survivor with a solution; let u be the earliest. If the better move dropped u, it cited an earlier survivor that the corrected argument proves solvable, contradicting the choice of u. □

Theorem B.2 (runs are sound)**.** *Let the filter be node-sound and return only playable customers. Suppose that at the start of a run the invariant holds, every memo entry is a genuine refutation, and every old move is refuted. Then the run leaves only genuine memo entries, and if it answers* false *at S, then $\neg\mathrm{Sol}_k(S)$. This holds for any child order and any memo policy, with the old move on or off (Appendix A, A.64).*

The proof is an induction over the run, in the order in which subtrees complete. Every old move a node holds is a sibling subtree that completed with *false* earlier in the same run, or an inherited one that passed the reinsertion test.

B.8 The split re-certification

Table B.1 gives the seven hardest re-certifications of Section 5.2 and the two values never certified before, both now certified by the same split. Each instance's tree was expanded at the top into independent subtrees (tasks), and each task was refuted by one worker. The seven re-certifications ended with every task refuted and the value unchanged. No task of any instance has found an order of the smaller value.

Table B.1: All seven hardest values re-certify unchanged, at 18 to 369 core-hours each. The two values never certified before, 21 for `Random-125-125-2-3_0` and 25 for `Random-125-125-2-2_0`, are now certified (read 12 October 2026). *Value*: the stored optimum; *k*: the value minus one, which must be refuted.

Instance	Value	*k*	State	Tasks refuted	Nodes	Core-hours
`Random-125-125-4-1_0`	57	56	refuted	669	4.97×10^{10}	18.1
`Random-125-125-2-4_0`	24	23	refuted	1,686	4.98×10^{11}	154.1
`Random-125-125-4-5_0`	46	45	refuted	616	7.96×10^{10}	26.1
`Random-125-125-4-2_0`	57	56	refuted	842	7.64×10^{10}	30.2
`Random-125-125-2-1_0`	24	23	refuted	4,096	1.30×10^{12}	366.5
`Random-125-125-2-5_0`	20	19	refuted	4,168	1.28×10^{12}	368.8
`Random-125-125-4-4_0`	51	50	refuted	2,504	4.12×10^{11}	148.8
`Random-125-125-2-3_0`	21	20	refuted	9,665	4.25×10^{12}	1127.8
`Random-125-125-2-2_0`	25	24	refuted	17,001	7.38×10^{12}	2230.6

C The bibliometric study

This appendix gives the method behind Section 3 in full, the counts per name and when each name was in use.

Search. We measured how much each name is used with OpenAlex (Priem et al., 2022), on 29 September 2026. For each of the twelve problems we searched titles and abstracts for its name and its common variants, and restricted the search to the four fields in which the problems are studied: Computer Science, Mathematics, Engineering and Decision Sciences. Without that restriction several names are ordinary words: "narrowness" returns over a million works, and "edge separation" returns aerodynamics.

Relevance. Inside those fields a search hit is still not a work that uses the name. OpenAlex's search stems words and ignores punctuation, so a quoted phrase matches more than the phrase. "Narrowness" matches "narrowing" in term rewriting; "node searching" matches look-ups in peer-to-peer networks; "split bandwidth" matches radar interferometry and 5G spectrum assignment; "path-width" matches the widths of roads, tool paths and datapaths. We therefore fetched all 2,271 hits with their titles, abstracts, topics and venues, and kept a work only if it is about the problem in the sense of the Linhares–Yanasse table. For the eleven smaller names the 700 hits were judged one by one. For pathwidth, the 1,571 hits were filtered by a rule: keep a work that uses the one-word spelling, that OpenAlex places in a theory subfield, or that mentions graphs,

Table C.1: Citations do not track usage: the most-cited papers are cited far more often than their problem's name is used. Works that use each problem's name in the sense of the Linhares–Yanasse table, and citations of the paper the table cites for it. *Relevant*: hits kept by the relevance check. *Citations*: OpenAlex `cited_by_count`, 29 September 2026. Kashiwabara and Fujisawa (1979) has no OpenAlex record. [a]OpenAlex returned one pathwidth record twice: the counts are of records, as Figure 3.1 draws them, and there are 1,212 distinct relevant pathwidth works.

Problem	Discipline	Relevant	Hits	Reference in the 2002 table	Citations
Pathwidth	graph theory	1,213[a]	1,571	Kinnersley (1992)	214
Gate matrix layout	VLSI	110	125	Möhring (1990); Wing et al. (1985)	134 / 89
PLA folding	VLSI	68	74	Möhring (1990)	134
MOSP	operations research	58	60	Yanasse (1997b); Fink and Voß (1999)	75 / 71
Vertex separation	graph theory	55	100	Kinnersley (1992)	214
Edge search	graph theory	38	95	Kirousis and Papadimitriou (1986)	293
Node search	graph theory	34	127	Kirousis and Papadimitriou (1985)	124
One-dimensional logic	VLSI	11	18	Ohtsuki et al. (1979)	108
Interval thickness	graph theory	6	10	Kashiwabara and Fujisawa (1979)	–
Narrowness	graph theory	0	35	Kornai and Tuza (1992)	44
Split bandwidth	graph theory	0	35	Fomin (1998)	19
Edge separation	graph theory	0	21	Lengauer (1981)	77
All twelve		1,593	2,271		

treewidth or minors and not roads, vehicles, trajectories or datapaths. We checked the rule by reading samples of what it kept and what it dropped. Of the 2,271 hits, 1,593 are relevant: 1,213 for pathwidth and 380 for the other eleven names. The two search-game names keep 27% and 40% of their hits.

Robustness. The counts are lower bounds. A work can study a problem without naming it in its abstract, OpenAlex has few abstracts before about 1990, and its citation counts run below Google Scholar's. The judgements were made by one annotator with model assistance, record a yes or no per work and not the reason, and have not been checked by a second annotator. As a second reading we applied a mechanical rule: the literal phrase with a context term and no exclusion term. It agrees with the judgements on 543 of the 700 judged works (78%). Under both readings pathwidth has more than three times as many works as the other eleven names together (3.2 and 3.6 times), the three empty names stay empty apart from one 5G false positive, and the search games lose most of their hits. MOSP's rank over all years is the one conclusion that depends on the reading: fourth under the judgements, fifth under the rule.

Counts and citations. Table C.1 gives the counts per name. Citation counts do not track name usage. Kirousis and Papadimitriou (1986) is the most-cited paper in the table (293 citations), while only 38 works use "edge search" in its sense. The paper is cited as a foundational result on graph searching, not because people work on the edge search game under that name. The

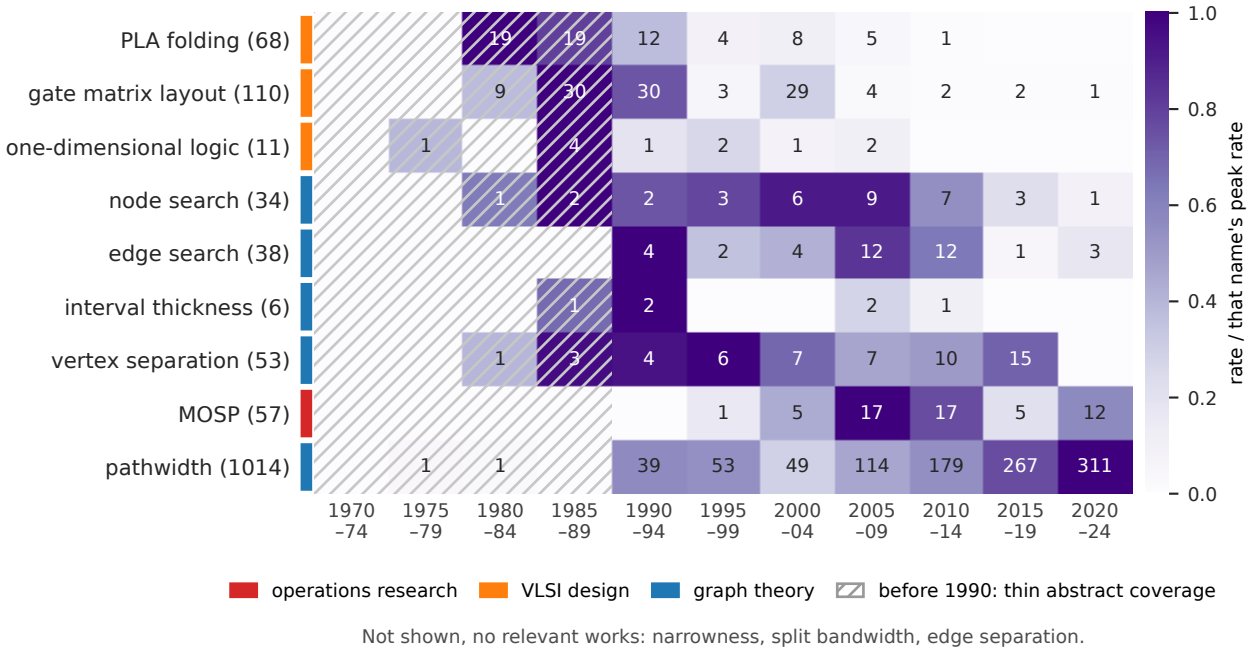


Figure C.1: The VLSI names belong to the 1980s, the search games peak later and fade, MOSP peaks in 2005–2014, and only pathwidth is still rising. The shade is the name's rate, relevant works per million works in the same four fields in each five-year period, as a fraction of that name's own peak rate; the numbers are raw counts. Rows are ordered by the period of their peak. Counts cover 1970–2024, since 2025–2026 is not fully indexed, so row totals are below those of Table C.1. Before 1990 (hatched) OpenAlex has few abstracts.

split is starker for Ohtsuki et al. (1979): 108 citations against 11 works using "one-dimensional logic". Split bandwidth is Fomin's own term in the paper the table cites (Fomin, 1998), and no one adopted it.

When each name was in use. Figure C.1 shows the counts over time. PLA folding and gate matrix layout peak in 1980–1989 and fall to almost nothing after 2010, and one-dimensional logic has no relevant work after 2009. MOSP first appears in the 1990s, peaks in 2005–2014 and is still active. Pathwidth's rate has roughly doubled since the 1990s, from about 15 to 27 works per million, and its 311 relevant works in 2020–2024 are more than the whole history of any other name. Rows with a few works per period move by whole cells on one paper, and should be read as anecdote rather than trend.

D A SAT formulation of MOSP

Our SAT formulation encodes the question "is $Z(M) \leq k$?", for a matrix M with customers C (rows) and patterns P (columns), $m = |P|$, as a formula in conjunctive normal form, and hands it to a SAT solver. It shares no code and no pruning rule with the customer search of Section 4, so it is an independent check on that search's answers. A satisfying assignment gives a production order that can be verified by simulation. A refutation can be logged as a DRAT proof and checked by an independent checker (drat-trim). Write P_c for the patterns of customer c, and

positions $t = 0, \ldots, m-1$.

Variables. $x_{p,t}$: pattern p is cut at position t. $y_{p,t}$: pattern p is cut at or before position t. $o_{c,t}$: the stack of customer c is open at position t. Two auxiliaries per customer and position, $\alpha_{c,t}$ (some pattern of c has been cut) and $\omega_{c,t}$ (all patterns of c have been cut).

Clauses.

1. *Permutation.* Every pattern takes at least one position and every position at least one pattern, $\bigvee_t x_{p,t}$ and $\bigvee_p x_{p,t}$; and at most one of each, by a ladder encoding, shown below.
2. *Prefixes.* $x_{p,t} \Rightarrow y_{p,t}$; $y_{p,t} \Rightarrow y_{p,t+1}$; $y_{p,t} \Rightarrow y_{p,t-1} \vee x_{p,t}$ for $t > 0$; and $y_{p,0} \Rightarrow x_{p,0}$.
3. *Open stacks.* A stack is open at t if one of its patterns is cut at t, $x_{p,t} \Rightarrow o_{c,t}$ for $p \in P_c$, or if some of its patterns are cut by t and some are not: $y_{p,t} \Rightarrow \alpha_{c,t}$ and $\omega_{c,t} \Rightarrow y_{p,t}$ for $p \in P_c$, and $\alpha_{c,t} \wedge \neg\omega_{c,t} \Rightarrow o_{c,t}$. These are $2|P_c| + 1$ clauses per customer and position instead of the $|P_c|(|P_c| - 1)$ of the direct pairwise form. Only these polarities are needed: no solution gains by setting $o_{c,t}$ true without cause.
4. *Width.* At most k of the $o_{c,t}$ are true at each position t, by a totalizer.
5. *Symmetry.* Reversing a production order leaves the number of open stacks at each step unchanged (Yanasse, 1997a, Proposition 1), so the pattern with the most customers may be placed in the first $\lfloor m/2 \rfloor + 1$ positions.

The ladder. To say that at most one of $z_1, \ldots, z_n$ is true, the ladder introduces auxiliaries $r_1, \ldots, r_n$, where r_i reads "the true literal, if any, comes after position i", and one slack literal s for "none is true". Its clauses are

$$r_{i+1} \Rightarrow r_i \ (1 \le i < n), \qquad z_1 \Leftrightarrow \neg r_1, \qquad z_i \Leftrightarrow r_{i-1} \wedge \neg r_i \ (1 < i \le n), \qquad s \Leftrightarrow r_n.$$

The first family makes $r_1, \ldots, r_n$ a ladder, true up to some point and false after it, so it has exactly one step; the channelling clauses make each z_i true exactly when the step is at i, and s true when there is no step. So at most one z_i is true, with $n + 1$ auxiliaries and about $4n$ clauses, linear in n, where the pairwise form needs $n(n-1)/2$ clauses. Our encoder emits this form through PySAT's `EncType.ladder`.

What is proved. The formula with clauses 1–4 is satisfiable exactly when some production order keeps at most k stacks open (Appendix A, A.65). The proof models the two cardinality constraints by their meaning, not by the clauses the ladder and totalizer encodings emit; both are standard and independent of MOSP. The symmetry clause 5 is justified by the reversal argument above but is not in the Lean model. That the encoder emits exactly these clauses is a reading of its source, not a proof.

On the instance of Figure 4.2. The formula has 31,598 variables and 109,923 clauses. Kissat 4.0.4 answers "satisfiable" at $k = 7$ in 9 s and at $k = 6$ in 56 s, with an order whose peak simulates to 6, and "unsatisfiable" at $k = 5$ in 158 s. CaDiCaL 1.9.5 refutes $k = 5$ in 53 s with a DRAT proof of 1.4 million lemmas (112 MB compressed), which drat-trim verifies in 44 s. So the optimum is 6, independently of both versions of the customer search, with a refutation that a third party can check. The repository holds the proof trimmed by drat-trim to the lemmas the refutation uses (27 MB compressed) and a script that rebuilds the formula from the instance and checks the proof in about 35 s (`paper1/data/false_refutation/drat/`).